MACHINE LEARNING INTERVIEW PREPARATION

LLM Training Foundations: Fine-Tuning, Alignment & Infrastructure

Distributed Training, SFT, PEFT, RLHF, DPO/GRPO & Production Alignment

Target Persona: Senior AI Engineer / Applied AI Engineer (~3 YOE)

Focus Areas: ZeRO/FSDP memory math, LoRA/QLoRA, RLHF/PPO, DPO/GRPO, model merging, training monitoring

Includes: Step-by-Step Numerical Hand Calculations, PyTorch Implementations, SVG/HTML Diagrams

Module 01: Fine-Tuning Fundamentals & Distributed Training Infrastructure

1. Introduction & Intuition

The Core Bottleneck

A pretrained base model — its architecture, attention mechanism, and KV cache behavior are covered in `01_llm_foundations` — is only a starting point. Turning it into something useful requires updating its weights via gradient descent, and *training* memory is a much larger burden than *inference* memory. At inference, you only need to hold the model weights (plus a KV cache). At training time, you additionally need gradients for every parameter, and — if using Adam, the standard optimizer for LLM training — two more full-precision buffers per parameter to track momentum and variance. A model that comfortably fits on a single GPU for inference can require 6-8x more memory just to fine-tune. The bottleneck is that full fine-tuning of a modern LLM routinely exceeds single-GPU VRAM, forcing a choice between distributing training across many GPUs or reducing the number of trainable parameters (the subject of Module 03).

High-Level Intuition

Think of training memory as a backpack you must carry for every parameter in the model. Inference only requires the "item itself" (the weight). Training adds a duplicate-weight-sized item for the gradient, plus two more duplicate-weight-sized items for Adam's momentum and variance trackers — and typically a full-precision master copy of the weight itself, since mixed-precision training keeps the "source of truth" copy in fp32 even while computing in bf16/fp16. None of this can be dropped without changing the optimizer or the fine-tuning strategy.

When one GPU's backpack won't fit, there are two different ways to lighten the load:

1. **Shard the backpack's contents** across a group of GPUs, so each GPU only carries a fraction of the gradients/optimizer-states/parameters (ZeRO, FSDP) — every GPU still sees every token, but only owns a slice of the model's "extra baggage."
2. **Shard the model itself** across GPUs, so each GPU only computes a slice of every layer (Tensor Parallelism) or owns a contiguous set of layers (Pipeline Parallelism) — different GPUs are responsible for different parts of the actual computation, not just different slices of the same computation's bookkeeping.

---

2. Core Concepts & Mathematical Formulation

Mixed-Precision Training Memory: The "16Ψ Bytes" Wall

Purpose & Intuition

This is the single most commonly asked number in LLM training system-design interviews. In standard mixed-precision training (bf16/fp16 compute, fp32 optimizer state — the convention popularized by the ZeRO paper), every parameter carries five separate buffers: an fp16/bf16 copy of the weight used for the forward/backward pass, an fp16/bf16 gradient, and three fp32 buffers (a master weight copy, Adam's first moment m, and Adam's second moment v). Knowing this breakdown lets you compute, from parameter count alone, whether a training job fits on your hardware before you ever launch it.

Mathematical Formulation

For a model with Ψ parameters:

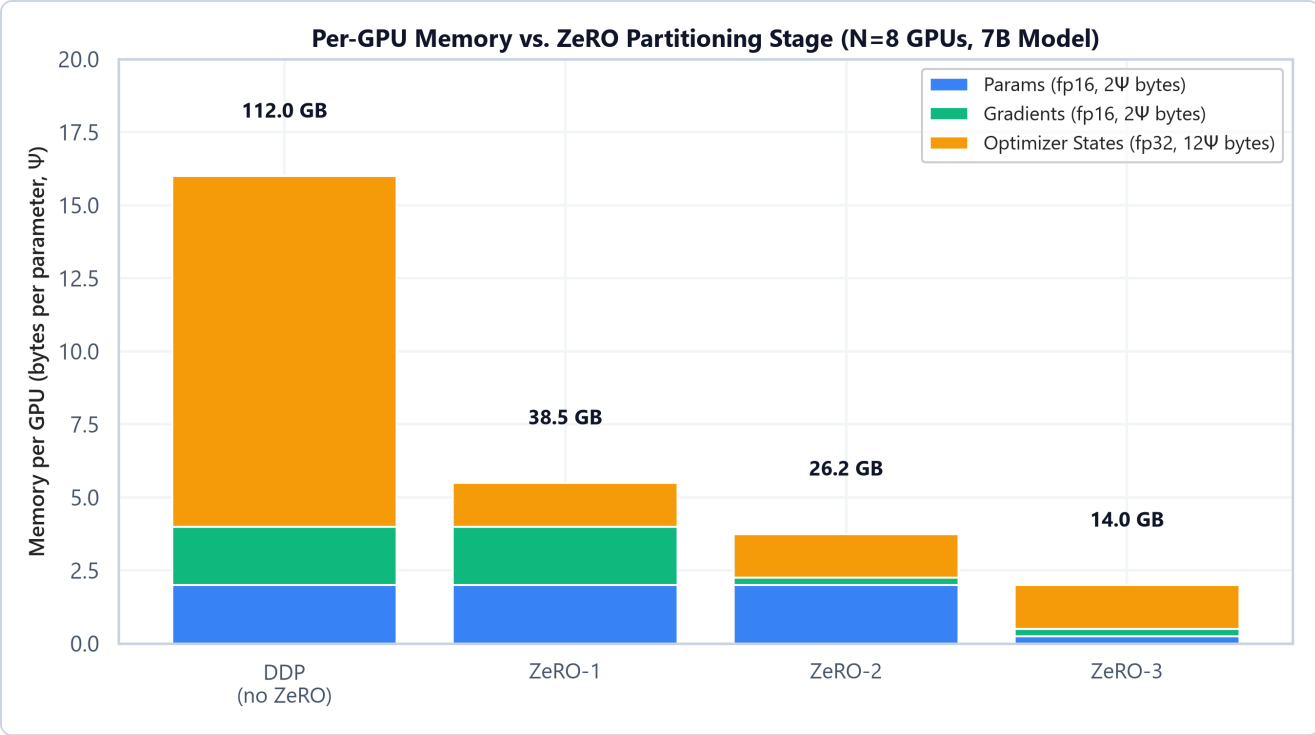
$$\text{Memory}_{\text{bytes}} = \underbrace{2\Psi}_{\text{fp16 params}} + \underbrace{2\Psi}_{\text{fp16 grads}} + \underbrace{4\Psi + 4\Psi + 4\Psi}_{\text{fp32 master + Adam } m,v} = 16\Psi \text{ bytes}$$

ZeRO: Partitioning the "16Ψ" Across GPUs

Intuition & Practical Use

Data-Parallel training (DDP) replicates the full 16Ψ bytes on every GPU — wasteful, since each GPU only ever needs its own optimizer-state shard to update its own parameter shard, not everyone else's. ZeRO (Zero Redundancy Optimizer) progressively shards this 16Ψ across N GPUs in three stages, trading a small amount of extra communication for a large reduction in per-GPU memory:

- **ZeRO-1:** Partitions only the optimizer states (the 12Ψ fp32 portion) across N GPUs.
- **ZeRO-2:** Additionally partitions the gradients (2Ψ) across N GPUs.
- **ZeRO-3:** Additionally partitions the parameters themselves (2Ψ) across N GPUs — every GPU only permanently owns $1/N$ of the model, all-gathering the rest just-in-time for each layer's forward/backward pass.



- **Plot Interpretation:** As ZeRO stage increases, more of the 16Ψ-byte total is divided by N instead of replicated. The params-only portion (2Ψ, blue) stays replicated until ZeRO-3, at which point it too shrinks by a factor of N, collapsing per-GPU memory from 16Ψ bytes down to just $16\Psi/N$ bytes.

Hand Calculation: 7B Model, N=8 GPUs

Let's compute per-GPU memory at each ZeRO stage for a 7B-parameter model ($\Psi = 7 \times 10^9$) sharded across $N = 8$ GPUs.

- **Step 1: Baseline (DDP, no ZeRO) — full 16Ψ replicated on every GPU**

$$\text{Memory}_{\text{DDP}} = 16 \times 7\text{B bytes} = 112 \text{ GB per GPU}$$

- **Step 2: ZeRO-1 — optimizer states (12Ψ) partitioned across 8 GPUs**

$$\text{Memory}_{\text{ZeRO-1}} = \left(2\Psi + 2\Psi + \frac{12\Psi}{8}\right) = 5.5\Psi \text{ bytes} = 5.5 \times 7\text{B} = 38.5 \text{ GB}$$

- **Step 3: ZeRO-2 — optimizer states and gradients (12Ψ + 2Ψ = 14Ψ) partitioned**

$$\text{Memory}_{\text{ZeRO-2}} = \left(2\Psi + \frac{14\Psi}{8}\right) = 3.75\Psi \text{ bytes} = 3.75 \times 7\text{B} = 26.25 \text{ GB}$$

- **Step 4: ZeRO-3 — everything (16Ψ) partitioned**

$$\text{Memory}_{\text{ZeRO-3}} = \frac{16\Psi}{8} = 2\Psi \text{ bytes} = 2 \times 7\text{B} = 14 \text{ GB}$$

Going from DDP to ZeRO-3 takes a 7B model's static training memory from 112 GB — infeasible on a single 80GB GPU — down to 14 GB, which comfortably fits, at the cost of additional all-gather/reduce-scatter communication for the sharded parameters.

Gradient Accumulation: Decoupling Batch Size from GPU Memory

Intuition & Practical Use

Larger batch sizes generally produce more stable gradient estimates, but a large batch may not fit in memory alongside the model, gradients, and optimizer states. Gradient accumulation runs several small "micro-batches" forward/backward, summing their gradients *without* stepping the optimizer, and only applies the optimizer step after the full accumulation window — simulating a much larger batch size without ever materializing it in memory at once.

Mathematical Formulation

$$B_{\text{eff}} = B_{\text{micro}} \times \text{accum\_steps} \times N_{\text{GPUs}}$$

Hand Calculation

With a micro-batch size of 4 sequences per GPU, 8 accumulation steps, and 4 GPUs:

$$B_{\text{eff}} = 4 \times 8 \times 4 = 128$$

Each GPU processes 4 sequences at a time — small enough to fit in memory — but the optimizer sees an effective batch of 128 sequences' worth of accumulated gradient signal before it takes a single step.

FSDP vs. ZeRO: Same Idea, Different Implementations

Both FSDP (PyTorch's Fully Sharded Data Parallel) and DeepSpeed's ZeRO shard optimizer state, gradients, and parameters across GPUs — conceptually, FSDP is very close to ZeRO-3. The differences that matter in practice:

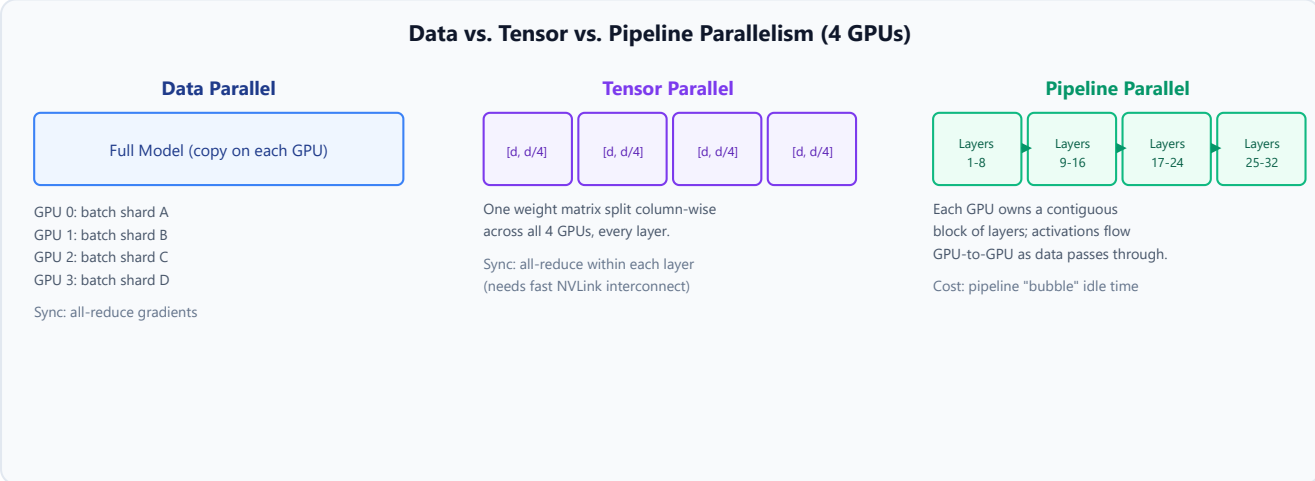
	ZeRO (DeepSpeed)	FSDP (PyTorch-native)
Ecosystem	Requires the DeepSpeed library	Built into PyTorch directly

	ZeRO (DeepSpeed)	FSDP (PyTorch-native)
Stage granularity	Explicit stages 1/2/3, independently configurable	Primarily full-sharding (ZeRO-3-equivalent), with configurable wrapping policies
Offload	Mature CPU/NVMe offload support	CPU offload supported, less mature than DeepSpeed's
When to reach for it	Already in a DeepSpeed/Megatron-DeepSpeed stack, or need fine-grained stage control or NVMe offload	Native PyTorch workflows (e.g. `torchtune`, plain PyTorch training loops) without adding a new dependency

Tensor & Pipeline Parallelism: Sharding the Model Itself

Unlike ZeRO/FSDP (which shard the *optimizer's bookkeeping* while every GPU still runs the full forward/backward computation), Tensor and Pipeline Parallelism shard the *computation*:

- **Tensor Parallelism (TP):** Splits individual weight matrices column-wise or row-wise across GPUs. For example, an FFN up-projection weight of shape `[d, 4d]` becomes `[d, 4d/N]` per GPU; each GPU computes its own partial output, and an all-reduce combines the partial results before the next layer. TP requires a fast interconnect (NVLink) since it synchronizes *within* every layer.
- **Pipeline Parallelism (PP):** Splits the model *by layer* — GPU 0 holds layers 1-8, GPU 1 holds layers 9-16, and so on — with activations passed between GPUs as data flows through the pipeline stages. This introduces a "bubble": GPUs downstream sit idle while waiting for the first micro-batch to arrive, and upstream GPUs sit idle waiting for the pipeline to drain at the end. Micro-batching (splitting each batch into smaller chunks that flow through the pipeline in an interleaved 1F1B schedule) is the standard mitigation.



Tensor & Shape Tracking

- **Full model weight matrix (e.g. FFN up-projection):** `[d, 4d]`
- **Tensor-Parallel shard (N-way column split):** `[d, 4d/N]` per GPU
- **Pipeline-Parallel per-stage parameter set:** `[num_layers / N_stages]` transformer blocks per GPU
- **Micro-batch (gradient accumulation):** `[B_micro, L, d]`, accumulated over `accum_steps` before an optimizer step processes the effective `[B_eff, L, d]` gradient signal

---

3. Implementation & Reference Code

Below is a self-contained memory-accounting utility that reproduces the hand calculations above, plus a minimal gradient-accumulation training loop.

```
import torch
import torch.nn as nn

def zero_stage_memory_gb(num_params: int, zero_stage: int, num_gpus: int) -> float:
    """Computes per-GPU training memory (GB) for mixed-precision Adam training at a given ZeRO stage."""
    params_bytes = 2 * num_params      # fp16/bf16 params
    grads_bytes = 2 * num_params       # fp16/bf16 grads
    optim_bytes = 12 * num_params      # fp32 master weight + Adam m + Adam v

    if zero_stage >= 3:
        params_bytes //= num_gpus
    if zero_stage >= 2:
        grads_bytes //= num_gpus
    if zero_stage >= 1:
        optim_bytes //= num_gpus

    total_bytes = params_bytes + grads_bytes + optim_bytes
    return total_bytes / 1e9  # GB


if __name__ == "__main__":
    num_params = 7_000_000_000  # 7B model
    num_gpus = 8

    for stage in [0, 1, 2, 3]:
        mem_gb = zero_stage_memory_gb(num_params, stage, num_gpus)
        label = "DDP (no ZeRO)" if stage == 0 else f"ZeRO-{stage}"
        print(f"{label:<16}: {mem_gb:6.2f} GB per GPU")

    # --- Gradient accumulation training loop ---
    torch.manual_seed(42)
    model = nn.Linear(16, 16)
```

```
    optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)

     B_micro, accum_steps, N_GPUS = 4, 8, 4  # matches the B_eff = 4 x 8 x 4 = 128
hand calc above
    optimizer.zero_grad()
    for step in range(accum_steps):
        x = torch.randn(B_micro, 16)          # [B_micro, d]
        target = torch.randn(B_micro, 16)     # [B_micro, d]
         loss = nn.functional.mse_loss(model(x), target) / accum_steps  # normalize
contribution
        loss.backward()  # accumulates .grad, does NOT step

      optimizer.step()   # single step after accumulating gradient from B_micro *
accum_steps samples
    optimizer.zero_grad()
     # This single-process demo only runs one GPU's worth of accumulation (B_micro *
accum_steps = 32);
      # in an actual N_GPUS=4 distributed run, each GPU accumulates independently
before an all-reduce,
    # so the true effective batch is scaled by N_GPUS as well, matching the hand calc
above.
    b_eff_single_gpu = B_micro * accum_steps
    b_eff_distributed = b_eff_single_gpu * N_GPUS
    print(f"\nEffective batch size (this single-GPU demo): {b_eff_single_gpu}")
         print(f"Effective  batch  size  (if  distributed  across  N_GPUS={N_GPUS}):
{b_eff_distributed}")
```

Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Fitting the training-time memory footprint (params + gradients + optimizer states, ~16 bytes/param) of a large model onto hardware where it would otherwise be infeasible on a single GPU.
- **Why Introduced over Legacy Approaches:** Naive Data-Parallel (DDP) training replicates the full optimizer state redundantly on every GPU. ZeRO/FSDP eliminate that redundancy by sharding, while TP/PP go further and shard the actual computation for models too large to fit even a single sharded replica's activations comfortably.
- **Key Failure Modes & Limitations:** ZeRO-3/FSDP introduce additional all-gather communication on every forward and backward pass, which can become bandwidth-bound on slow interconnects. Pipeline Parallelism's bubble overhead wastes GPU-time proportional to the number of pipeline stages if micro-batching isn't tuned well.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Sharding strategies (ZeRO/FSDP/TP/PP) do not change the total FLOPs of training — they change *where* those FLOPs execute and how much *communication* is needed to keep GPUs fed.
- **Space/Memory Footprint:** DDP: 16Ψ bytes replicated per GPU. ZeRO-k: shrinks the corresponding portion of 16Ψ by a factor of N. TP: shrinks per-GPU *activation and weight* memory for the sharded layers by roughly $1/N_{\text{TP}}$. PP: shrinks per-GPU *parameter* memory by roughly $1/N_{\text{stages}}$.
- **Primary Bottleneck Type:** Memory-bandwidth/interconnect-bound for ZeRO-3/FSDP (frequent all-gather) and Tensor Parallelism (per-layer all-reduce); compute-bound within each GPU's local matrix multiplications; pipeline-bubble-bound (wasted idle time) for Pipeline Parallelism at small micro-batch counts.
- **Variable Legend:** Ψ = parameter count, N = number of GPUs, B_{micro} = micro-batch size, B_{eff} = effective batch size, d = hidden dimension.

3. Production & Scalability

- **Deployment Considerations:** Most production LLM training stacks combine strategies (e.g., ZeRO-3 within a node + data parallelism across nodes, or 3D parallelism combining TP + PP + data parallelism for the largest models) rather than picking a single one in isolation.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: You have 8 GPUs and a 7B model that needs 112GB unsharded. Which ZeRO stage is the minimum needed to fit on 80GB GPUs, and why?**
>
> ZeRO-1 already gets it to 38.5GB per GPU, comfortably under 80GB. The interviewer signal here is recognizing you don't need to jump straight to ZeRO-3 (which adds the most communication overhead) if a lighter stage already fits — match the stage to the actual memory constraint, not maximal sharding by default.

> **Question: Why does gradient accumulation not increase peak memory usage, even though it simulates a larger batch?**
>
> Because each micro-batch is fully processed (forward, backward, gradients accumulated into the existing `.grad` buffers) and its activations freed before the next micro-batch starts. Peak memory is bounded by one micro-batch's activation footprint, not the effective batch's — only the accumulated gradient buffer (already sized for the full parameter count regardless) persists across steps.

Module 02: Supervised Fine-Tuning (SFT) & Instruction Tuning

1. Introduction & Intuition

The Core Bottleneck

A raw pretrained model is a next-token predictor trained on raw web text — it completes "The capital of France is" fluently, but has no learned notion of "answer this instruction helpfully and stop." Ask it a direct question and it's just as likely to continue the prompt with more questions, or ramble into unrelated text, as it is to answer. The bottleneck is behavioral, not architectural: the base model already contains the knowledge needed to answer well, but has never been trained on the specific *format* of "instruction in, helpful response out, then stop."

High-Level Intuition

Supervised Fine-Tuning (SFT) is showing the model thousands of examples of exactly that format — (instruction, ideal response) pairs — and training it with the same next-token-prediction objective as pretraining, but now over curated conversational data instead of raw web text. Critically, we only want to train the model to *generate* good responses, not to *memorize* the instructions themselves — so the loss is computed only over the response tokens, with the prompt tokens masked out of the loss entirely.

---

2. Core Concepts & Mathematical Formulation

SFT Loss with Prompt-Token Masking

Purpose & Intuition

If we computed the standard causal LM loss over the *entire* sequence (prompt + response), the model would waste capacity learning to predict the prompt tokens themselves — which are given as input, not something the model should learn to generate. Masking the loss to only the response tokens focuses every gradient update on what actually matters: producing a good completion given the instruction.

Mathematical Formulation

For a tokenized sequence of length L with a binary mask $m_i \in \{0, 1\}$ (1 for response tokens, 0 for prompt tokens):

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{\sum_i m_i} \sum_{i=1}^{L} m_i \cdot \log P(w_i \mid w_{<i})$$

The mask ensures the denominator only counts response tokens, so the average loss is not diluted by (and no gradient flows from) the prompt portion.

Hand Calculation: Masked Loss on a Tiny Sequence

Let's compute the masked SFT loss for a toy 6-token sequence: prompt = `["Summarize:", "cats", "sleep"]` (mask = 0), response = `["Cats", "nap", "often"]` (mask = 1).

- **Step 1: Per-token negative log-probabilities** (hypothetical model outputs)
  Assume the model assigns these $-\log P(w_i \mid w_{<i})$ values to each position:
Token	Mask	$-\log P$
"Summarize:"	0	2.1
"cats"	0	1.8
"sleep"	0	3.0
"Cats"	1	0.9
"nap"	1	1.4
"often"	1	0.6

- **Step 2: Zero out the masked (prompt) positions' contribution**
  Only the response row values (0.9, 1.4, 0.6) enter the sum; the prompt rows (2.1, 1.8, 3.0) are excluded entirely, not just down-weighted.

- **Step 3: Average over response tokens only**

$$\mathcal{L}_{\text{SFT}} = \frac{0.9 + 1.4 + 0.6}{3} = \frac{2.9}{3} \approx 0.9667$$

Note the denominator is 3 (the number of response tokens), not 6 (the full sequence length) — this is the entire point of masking: the loss magnitude reflects only how well the model predicts the response, unaffected by how "surprising" the prompt tokens were.

Tensor & Shape Tracking

- **Input token IDs:** `[B, L]`
- **Loss mask:** `[B, L]` (binary, 1 for response tokens)
- **Logits:** `[B, L, V]`
- **Per-token loss (before masking):** `[B, L]`
- **Masked scalar loss:** `[]` (sum of masked per-token losses, divided by mask sum)

Instruction Dataset Construction & Chat Templates

Intuition & Practical Use

Instruction data is formatted with a **chat template** — a fixed set of special tokens/markers (e.g., `<|user|>`, `<|assistant|>`) that structure a conversation into a single flat token sequence the model can learn to parse positionally. Multi-turn conversations extend this pattern, with the loss mask covering *every* assistant turn's tokens (not just the final one), so the model learns to respond well at any point in a multi-turn exchange, not only as the last turn.

Data Quality vs. Quantity

A relatively small set (thousands, not millions) of high-quality, diverse instruction examples reliably outperforms a much larger but noisier or repetitive set — SFT is teaching *format and behavior*, not new knowledge, so the marginal value of additional examples drops quickly once the format is well-represented across enough different task types.

Synthetic Instruction-Data Generation

Intuition & Practical Use

Manually writing thousands of high-quality (instruction, response) pairs is expensive. A common alternative is generating them synthetically — prompting a strong existing LLM to produce instruction/response pairs, optionally seeded from a small set of human-written examples (self-instruct-style bootstrapping), then filtering aggressively before using them for SFT.

Quality Control Considerations

- **Diversity:** Synthetic generation left unchecked tends to collapse onto a narrow set of phrasing patterns and topics; explicit diversity sampling (varying instruction types, topics, and difficulty) is required to avoid a homogeneous dataset.
- **Deduplication:** Near-duplicate synthetic examples (common when sampling repeatedly from the same generator) waste training compute and can bias the model toward over-represented patterns; embedding-based or n-gram-based deduplication is standard.
- **Contamination:** Synthetic data generated by a model that was itself trained (even indirectly) on benchmark data can leak benchmark-like content into your SFT set, inflating eval scores in a way that doesn't reflect real generalization — decontamination against your eval suite is a required step, not optional hygiene.
- **Synthetic-Specific Risks:** Models generating synthetic data can propagate their own stylistic quirks, factual errors, or refusal patterns into the student model (a distillation-like effect) — synthetic data should be spot-checked for quality, not assumed correct because it's fluent.

---

3. Implementation & Reference Code

Below is a self-contained masked cross-entropy loss implementation and a chat-template formatting example.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

def sft_masked_loss(logits: torch.Tensor, targets: torch.Tensor, loss_mask: torch.Tensor) -> torch.Tensor:
    """Computes SFT cross-entropy loss, averaged only over response (mask=1) tokens.

    logits:    [B, L, V]
    targets:   [B, L]
    loss_mask: [B, L], 1.0 for response tokens, 0.0 for prompt tokens
    """
    B, L, V = logits.shape
    # .reshape() rather than .view(): a common real usage pattern is calling this on
    # a slice (e.g. logits[:, :-1, :] for next-token prediction), which is not
    # contiguous in memory -- .view() throws RuntimeError there, .reshape() copies
    # transparently when needed and works on both fresh and sliced tensors.
    per_token_loss = F.cross_entropy(
        logits.reshape(B * L, V), targets.reshape(B * L), reduction="none"
    ).reshape(B, L)  # [B, L]

    masked_loss = per_token_loss * loss_mask  # [B, L]
    return masked_loss.sum() / loss_mask.sum().clamp(min=1)  # scalar


def format_chat_example(instruction: str, response: str) -> tuple[str, list[int]]:
    """Builds a single flat chat-formatted string and a token-level loss mask marker
    (illustrated at the string level; a real tokenizer would map this to token-level masks)."""
    prompt_part = f"<|user|>\n{instruction}\n<|assistant|>\n"
    response_part = f"{response}<|end|>"
    full_text = prompt_part + response_part
      # 0 for every prompt character's "region", 1 for every response character's "region" (illustrative only)
    mask_marker = [0] * len(prompt_part) + [1] * len(response_part)
```

```
    return full_text, mask_marker


if __name__ == "__main__":
    torch.manual_seed(42)
    B, L, V = 2, 6, 1000

    logits = torch.randn(B, L, V)
    targets = torch.randint(0, V, (B, L))
    # First 3 tokens are prompt (masked out), last 3 are response (kept)
    loss_mask = torch.tensor([[0., 0., 0., 1., 1., 1.],
                              [0., 0., 1., 1., 1., 1.]])

    loss = sft_masked_loss(logits, targets, loss_mask)
    print(f"Masked SFT loss: {loss.item():.4f}")

    text, mask = format_chat_example("Summarize: cats sleep", "Cats nap often")
    print(f"\nFormatted example:\n{text}")
      print(f"Prompt region length: {mask.count(0)} chars, Response region length:
{mask.count(1)} chars")
```

Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Converting a raw next-token-prediction base model into one that follows the instruction-response format and behaves helpfully when prompted directly.
- **Why Introduced over Legacy Approaches:** Prompt-engineering a base model (few-shot examples in-context) can partially elicit instruction-following behavior, but SFT bakes the behavior into the weights directly, producing much more reliable zero-shot instruction-following without needing few-shot examples at inference time.
- **Key Failure Modes & Limitations:** Catastrophic forgetting — aggressive fine-tuning on a narrow instruction distribution can degrade the base model's broader pretraining knowledge and capabilities if the dataset lacks diversity or the learning rate is too high.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Identical to pretraining's forward/backward FLOPs per token — SFT doesn't change the per-token compute cost, only the data distribution and (via masking) which tokens contribute gradient signal.
- **Space/Memory Footprint:** Full-parameter SFT carries the full training memory footprint discussed in Module 01 (the "16Ψ bytes" wall) unless combined with the parameter-efficient methods in Module 03.

- **Primary Bottleneck Type:** Data-quality-bound more than compute-bound in practice — the ceiling on SFT quality is usually the diversity and correctness of the instruction dataset, not GPU throughput.
- **Variable Legend:** L = sequence length, V = vocabulary size, m_i = loss mask at position i.

3. Production & Scalability

- **Deployment Considerations:** SFT datasets are typically versioned and evaluated independently of the model itself (dataset ablations), since swapping in a higher-quality instruction set often improves downstream quality more than architectural changes at this stage.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Why mask the loss on prompt tokens instead of just training on the full sequence?**
>
> The prompt is given as input at inference time, not generated — training the model to predict it wastes gradient signal on a task it will never need to perform, and can dilute the loss magnitude on the tokens that actually matter (the response).

> **Question: What's the risk of using purely synthetic data generated by another LLM for SFT?**
>
> Without careful filtering, synthetic data can be low-diversity (collapsing onto the generator model's stylistic patterns), contain factual errors the generator itself has, or leak benchmark-like content if the generator was exposed to eval data — all of which can silently degrade or mis-represent the fine-tuned model's real capabilities.

Module 03: Parameter-Efficient Fine-Tuning (LoRA, QLoRA, Adapters)

1. Introduction & Intuition

The Core Bottleneck

Module 01 established that full fine-tuning of a 7B model costs roughly 112GB of training memory even before activations — and that number scales linearly with model size, reaching well past a single high-end GPU for anything in the tens-of-billions-of-parameters range. Distributed training (ZeRO/FSDP/TP/PP) solves this by adding *more hardware*. Parameter-Efficient Fine-Tuning (PEFT) solves it from the opposite direction: instead of training all Ψ parameters, freeze the pretrained weights entirely and train a much smaller set of *new* parameters injected into the model — shrinking the optimizer-state memory (the dominant cost) by orders of magnitude, without needing extra GPUs at all.

High-Level Intuition

The key empirical observation behind LoRA (Low-Rank Adaptation) is that the *change* a model's weights need to undergo during fine-tuning — the delta between the pretrained weight and the ideally-fine-tuned weight — tends to have low "intrinsic rank." Rather than learning a full, dense update matrix the same size as the original weight, LoRA learns that update as the product of two much smaller matrices, $B \times A$, whose rank r is a tiny fraction of the original dimension. The frozen base weight stays untouched; only the small B and A matrices are trained, and at inference they can be merged back into the base weight ($W' = W + BA$) with zero added latency.

---

2. Core Concepts & Mathematical Formulation

LoRA: Low-Rank Weight Decomposition

Purpose & Intuition

For a frozen weight matrix $W \in \mathbb{R}^{d \times d}$, LoRA represents the *fine-tuning update* as $\Delta W = BA$, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times d}$, with rank $r \ll d$. Instead of training d^2 parameters per weight matrix, LoRA trains only $2rd$ — and because r is typically 8-64 while d is in the thousands, this is a 100-1000x reduction in trainable parameters for that matrix.

Mathematical Formulation

$$W' = W + \Delta W = W + BA, \qquad \text{params}_{\text{LoRA}} = 2rd \quad \text{vs.} \quad \text{params}_{\text{full}} = d^2$$

A is initialized with small random values (or zero) and B is initialized to zero, so that $\Delta W = 0$ at the start of training — the adapted model is numerically identical to the base model before any training happens, guaranteeing training starts from the pretrained model's exact behavior.

Hand Calculation: LoRA Trainable Parameters vs. Full Fine-Tuning

Let's compute the trainable-parameter savings for a single attention projection matrix with $d = 4096$ and LoRA rank $r = 8$.

- **Step 1: Full fine-tuning parameter count for this one matrix**

$$\text{params}_{\text{full}} = d^2 = 4096^2 = 16{,}777{,}216 \approx 16.8\text{M params}$$

- **Step 2: LoRA trainable parameter count for the same matrix**

$$\text{params}_{\text{LoRA}} = 2rd = 2 \times 8 \times 4096 = 65{,}536 \approx 0.066\text{M params}$$

- **Step 3: Compute the reduction ratio**

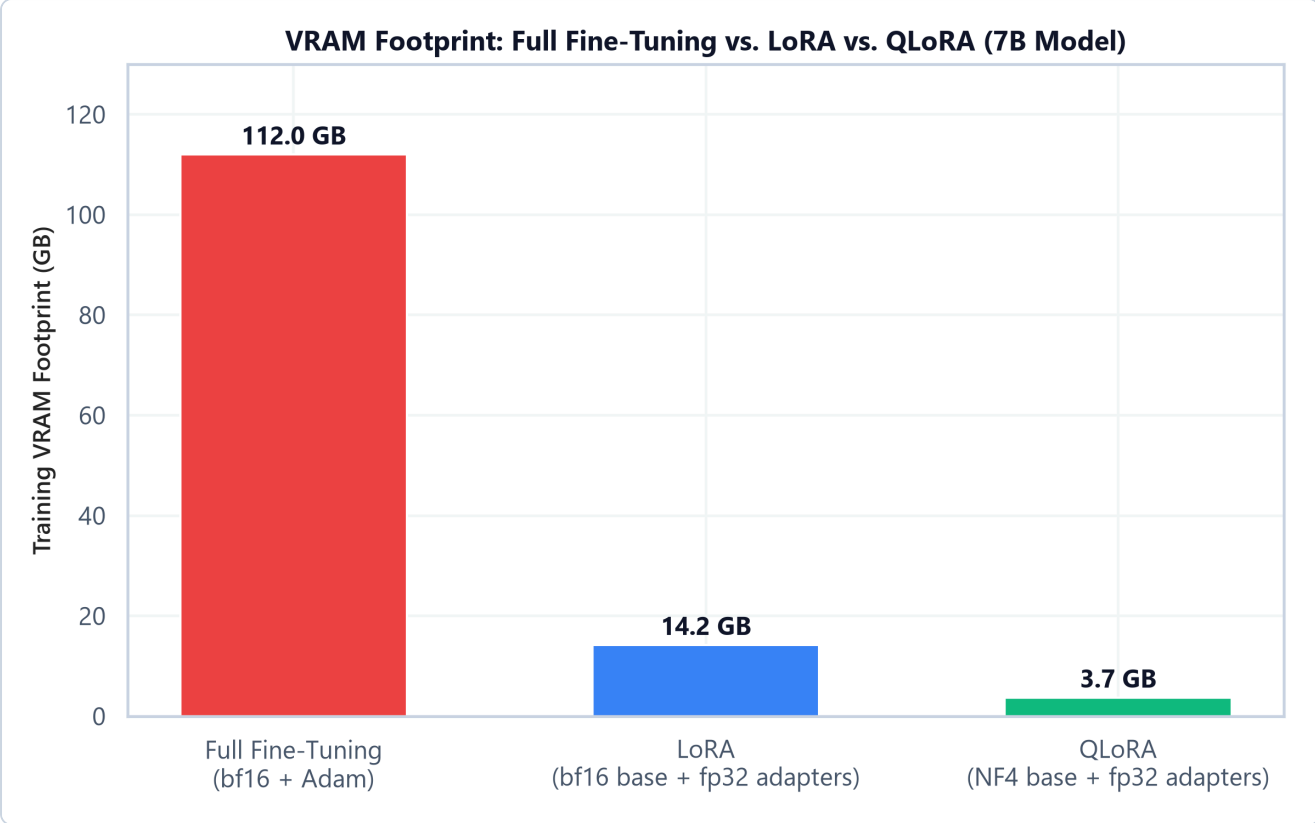
$$\frac{\text{params}_{\text{full}}}{\text{params}_{\text{LoRA}}} = \frac{16{,}777{,}216}{65{,}536} = 256\times$$

For this one matrix, LoRA trains 256x fewer parameters than full fine-tuning — and since the optimizer state (Adam's m, v, and fp32 master copy) is only allocated for *trainable* parameters, this 256x reduction applies directly to the dominant memory cost from Module 01, not just to parameter count.

QLoRA: Quantized Base Weights + LoRA Adapters

Purpose & Intuition

LoRA already freezes the base weights, but they still need to be stored in memory (typically bf16, 2 bytes/param) to run the forward pass. QLoRA goes further: it quantizes the *frozen* base weights down to 4-bit precision (NF4, a data type tuned for the roughly-normal distribution of neural network weights), while keeping the small trainable LoRA adapters in higher precision (bf16/fp32). Because the base weights are frozen and only used for the forward pass (dequantized on-the-fly for each matrix multiply), the 4-bit precision loss has a much smaller effect on training quality than it would if those weights were also being updated.



- **Plot Interpretation:** Full fine-tuning of a 7B model requires the full 16-bytes/param training memory (112GB). LoRA drops this to just the base model's inference-time footprint (bf16, ~14GB) plus a negligible adapter optimizer-state cost. QLoRA compresses the base weights further to 4-bit (NF4), cutting the base footprint to roughly a quarter of LoRA's — under 4GB total for a 7B model, small enough to fine-tune on a single consumer GPU.

Hand Calculation: QLoRA Base-Weight Memory Savings

For the same 7B-parameter base model, compare 16-bit vs. 4-bit (NF4) storage of the *frozen* weights.

- **Step 1: 16-bit (bf16) base weight storage**

$$\text{Memory}_{\text{bf16}} = 2 \text{ bytes} \times 7\text{B params} = 14 \text{ GB}$$

- **Step 2: 4-bit (NF4) base weight storage**

$$\text{Memory}_{\text{NF4}} = 0.5 \text{ bytes} \times 7\text{B params} = 3.5 \text{ GB}$$

- **Step 3: Compute the reduction**

$$\text{Memory}_{\text{bf16}} - \text{Memory}_{\text{NF4}} = 14 \text{ GB} - 3.5 \text{ GB} = 10.5 \text{ GB saved}$$

Combined with LoRA's tiny trainable-adapter footprint, QLoRA brings a 7B model's total fine-tuning memory down to under 4GB — a reduction from 112GB (full fine-tuning) of roughly 30x.

Tensor & Shape Tracking

- **Frozen base weight W:** `[d, d]` (bf16 for LoRA, NF4-packed for QLoRA)
- **LoRA down-projection A:** `[r, d]`
- **LoRA up-projection B:** `[d, r]`
- **LoRA update $\Delta W = BA$:** `[d, d]` (same shape as W, computed on-the-fly, never materialized densely during training)
- **Forward pass output:** `[B, L, d]` = `x @ (W + BA).T`, equivalently `x @ W.T + (x @ A.T) @ B.T`

Adapters, Prefix Tuning, and LoRA Target Module Selection

Adapter Layers vs. LoRA

Adapter layers insert small bottleneck feed-forward blocks (down-project → nonlinearity → up-project) *between* existing transformer layers, adding extra sequential computation at inference time. LoRA instead reparameterizes an *existing* weight matrix's update and — because BA can be merged directly into W after training — adds zero extra inference latency, which is why LoRA has become the more widely adopted default over classic adapters for LLM fine-tuning.

Prefix / Prompt Tuning

Prefix Tuning prepends a small number of trainable "virtual token" embeddings to the input at every layer, steering the model's behavior without touching any existing weights at all. It trains even fewer parameters than LoRA in many configurations, but tends to be more sensitive to initialization and harder to optimize than LoRA in practice.

LoRA Target Module Selection

LoRA can be applied to any weight matrix, but not all choices are equal:

Target modules	Trainable params	Typical effect
`q_proj`, `v_proj` only	Smallest	Original LoRA paper's default; captures most of the benefit cheaply
`q_proj`, `k_proj`, `v_proj`, `o_proj`	Small-medium	Adapts the full attention block, often a small quality gain over Q/V-only

Target modules	Trainable params	Typical effect
Attention + FFN (`gate_proj`, `up_proj`, `down_proj`)	Largest (still ≪ full FT)	Closest to full fine-tuning quality, at a proportionally higher (but still small) parameter/memory cost

The trade-off is monotonic but has diminishing returns: extending LoRA to FFN layers usually narrows the quality gap to full fine-tuning further, at a parameter cost still orders of magnitude below full fine-tuning — the right choice depends on how much of that remaining gap matters for the task.

---

3. Implementation & Reference Code

Below is a self-contained PyTorch implementation of a LoRA-wrapped linear layer, verified against the trainable-parameter hand calculation above.

```
import torch
import torch.nn as nn

class LoRALinear(nn.Module):
    def __init__(self, in_features: int, out_features: int, rank: int = 8, alpha: int
= 16):
        super().__init__()
        self.base = nn.Linear(in_features, out_features, bias=False)
        self.base.weight.requires_grad_(False)  # freeze the pretrained weight

           self.lora_A = nn.Parameter(torch.randn(rank, in_features) * 0.01)   # [r,
d_in]
        self.lora_B = nn.Parameter(torch.zeros(out_features, rank))        # [d_out,
r]
        self.scaling = alpha / rank

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: [B, L, d_in]
        base_out = self.base(x)                                    # [B, L, d_out]
        lora_out = (x @ self.lora_A.T) @ self.lora_B.T              # [B, L, d_out]
        return base_out + self.scaling * lora_out                   # [B, L, d_out]

    def count_trainable_params(self) -> int:
        return self.lora_A.numel() + self.lora_B.numel()


if __name__ == "__main__":
    torch.manual_seed(42)
    d, r = 4096, 8
    layer = LoRALinear(in_features=d, out_features=d, rank=r)
```

```
    full_ft_params = d * d
    lora_params = layer.count_trainable_params()
    print(f"Full fine-tuning params for this matrix: {full_ft_params:,}")
    print(f"LoRA trainable params (r={r}):           {lora_params:,}")
      print(f"Reduction factor:                                {full_ft_params /
lora_params:.0f}x")

    x = torch.randn(2, 10, d)  # [B=2, L=10, d=4096]
    out = layer(x)
    print(f"\nOutput shape: {tuple(out.shape)} (unchanged by LoRA, as expected)")

     # Confirm zero-initialization: with lora_B == 0, output must exactly equal the
frozen base layer
     assert torch.allclose(out, layer.base(x)), "LoRA output should match base model
at initialization"
    print("Verified: LoRA output matches frozen base model exactly before training (B
initialized to 0).")
```

Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Reducing trainable-parameter count (and therefore optimizer-state memory) by orders of magnitude, making fine-tuning of large models feasible on a single GPU without any distributed training infrastructure.
- **Why Introduced over Legacy Approaches:** Earlier adapter methods added inference-time latency by inserting new sequential layers. LoRA's reparameterization can be merged directly into the frozen weight after training, making it functionally free at inference — a major reason it displaced classic adapters as the default PEFT method.
- **Key Failure Modes & Limitations:** Too small a rank r can underfit tasks that require substantial behavioral change from the base model; QLoRA's 4-bit quantization introduces a small but nonzero error that compounds if the base model is quantized multiple times in a pipeline (e.g., quantize → fine-tune → merge → re-quantize).

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** LoRA adds a small amount of extra compute per forward pass ($2 \times B \times L \times r \times d$ for the low-rank path) on top of the frozen base matmul — negligible relative to the base model's own FLOPs since $r \ll d$.
- **Space/Memory Footprint:** Trainable parameters and their optimizer states scale with $2rd$ per adapted matrix instead of d^2; QLoRA additionally reduces the frozen base weight storage from 2 bytes/param (bf16) to roughly 0.5 bytes/param (NF4).

- **Primary Bottleneck Type:** Compute-bound on the (tiny) low-rank matmuls; the frozen base weights remain memory-bandwidth-bound to load for the forward pass, same as inference.
- **Variable Legend:** d = hidden dimension, r = LoRA rank, α = LoRA scaling factor, Ψ = total parameter count.

3. Production & Scalability

- **Deployment Considerations:** LoRA adapters can be merged into the base weights for zero-latency serving, or kept unmerged and swapped per-request to serve many fine-tuned "personalities" from one base model in memory (see Module 06 for multi-adapter composition and routing).

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Why is `lora*B*` *initialized to zero while* `*lora*A` is initialized with small random values, rather than both being zero or both being random?**
>
> Initializing B to zero guarantees $\Delta W = BA = 0$ at the start of training regardless of A's values, so the adapted model exactly reproduces the pretrained model's behavior before any gradient updates — a safe, deterministic starting point. If both were zero, gradients through A would also be zero (since $\partial \mathcal{L}/\partial A$ depends on B), stalling training entirely.

> **Question: Why does QLoRA quantize only the base weights and not the LoRA adapters themselves?**
>
> The base weights are frozen and only read during the forward pass, so quantization error there is a fixed, bounded perturbation. The LoRA adapters are what's actually being trained — quantizing parameters that need precise gradient updates would directly degrade training quality, so they're kept in higher precision (bf16/fp32).

Module 04: Reward Modeling & Reinforcement Learning from Human Feedback (RLHF)

1. Introduction & Intuition

The Core Bottleneck

SFT (Module 02) teaches a model to imitate example responses, but imitation has a ceiling: it can only be as good as the demonstrations it was trained on, and it provides no signal about *which of several plausible responses is better* — only "produce something like this one." Many desirable qualities (helpfulness, harmlessness, following subtle instructions precisely) are far easier for a human to *judge by comparison* ("response A is better than response B") than to *demonstrate* by writing a single ideal response from scratch. The bottleneck is that SFT has no mechanism to learn from comparative preference judgments at all.

High-Level Intuition

RLHF closes this gap in two stages. First, train a **reward model** — a separate model that takes a (prompt, response) pair and outputs a scalar score — using human preference comparisons (given two responses to the same prompt, which one do people prefer?). Second, use that reward model as a learned objective to further train the SFT model via reinforcement learning (PPO): generate a response, score it with the reward model, and nudge the policy's weights to make higher-scoring responses more likely — while a KL-divergence penalty keeps the policy from drifting too far from its SFT starting point, which would otherwise let it find degenerate outputs that score well on the reward model but are no longer coherent or safe.

---

2. Core Concepts & Mathematical Formulation

Reward Model Training: The Bradley-Terry Preference Loss

Purpose & Intuition

Human preference data comes in the form of pairwise comparisons: given a prompt and two candidate responses, a labeler picks the preferred one. The Bradley-Terry model converts this pairwise-preference signal into a training objective for a scalar reward model r_θ: the model should assign a *higher* score to the preferred response than the rejected one, and the loss penalizes it proportionally to how wrong (or barely right) that ordering is.

Mathematical Formulation

For a preferred response y_w ("winner") and rejected response y_l ("loser") to the same prompt x:

$$\mathcal{L}_{\text{RM}} = -\log \sigma\big(r_\theta(x, y_w) - r_\theta(x, y_l)\big)$$

where σ is the sigmoid function. This loss is minimized when $r_\theta(x, y_w) \gg r_\theta(x, y_l)$ (confidently correct ordering) and grows large when the model scores the rejected response higher than the preferred one.

Hand Calculation: Reward Model Loss for One Preference Pair

Suppose the reward model assigns these scores to two candidate responses for the same prompt:

- Preferred response: $r_\theta(x, y_w) = 2.4$
- Rejected response: $r_\theta(x, y_l) = 0.9$
- **Step 1: Compute the score difference**

$$\Delta r = r_\theta(x, y_w) - r_\theta(x, y_l) = 2.4 - 0.9 = 1.5$$

- **Step 2: Apply the sigmoid**

$$\sigma(1.5) = \frac{1}{1 + e^{-1.5}} \approx \frac{1}{1 + 0.2231} \approx 0.8176$$

- **Step 3: Compute the loss**

$$\mathcal{L}_{\text{RM}} = -\log(0.8176) \approx 0.2014$$

The loss is small because the model already scores the preferred response comfortably higher. If the scores were reversed ($r_\theta(x, y_w) = 0.9$, $r_\theta(x, y_l) = 2.4$, i.e. $\Delta r = -1.5$), $\sigma(-1.5) \approx 0.1824$ and $\mathcal{L}_{\text{RM}} = -\log(0.1824) \approx 1.7014$ — a much larger loss, correctly penalizing the wrong ordering.

PPO: The RLHF Training Objective

Purpose & Intuition

Proximal Policy Optimization (PPO) updates the policy (the LLM being fine-tuned) to increase the probability of actions (generated tokens/responses) that scored well under the reward model — but it *clips* how large a single update can be, preventing the kind of large, destabilizing policy shift that plain policy-gradient methods are prone to. A separate KL-divergence penalty against the frozen SFT reference policy further discourages the model from drifting into degenerate, reward-hacking outputs that score well on the reward model but no longer resemble coherent language.

Mathematical Formulation

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}\Big[\min\left(\rho_t \cdot \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \cdot \hat{A}_t\right)\Big] - \beta \cdot D_{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$$

where $\rho_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$ is the probability ratio between the updated and previous policy for the action taken, $\hat{A}_t$ is the estimated advantage (roughly, "how much better than average was this action"), ϵ is the clip range (commonly 0.1-0.2), and β controls the strength of the KL penalty against the reference policy π_{ref} (the frozen SFT model).

Hand Calculation: PPO Clipped Objective for One Token

Let's compute the clipped surrogate objective for a single token with an estimated advantage $\hat{A}_t = 1.2$ (this action was better than average), a probability ratio $\rho_t = 1.35$, and clip range $\epsilon = 0.2$.

- **Step 1: Compute the unclipped term**

$$\rho_t \cdot \hat{A}_t = 1.35 \times 1.2 = 1.62$$

- **Step 2: Compute the clipped ratio**

$$\text{clip}(1.35,\ 1-0.2,\ 1+0.2) = \text{clip}(1.35,\ 0.8,\ 1.2) = 1.2 \quad (1.35 \text{ exceeds the upper bo}$$

- **Step 3: Compute the clipped term**

$$\text{clip}(\rho_t, 0.8, 1.2) \cdot \hat{A}_t = 1.2 \times 1.2 = 1.44$$

- **Step 4: Take the minimum of the two terms**

$$\mathcal{L}_{\text{PPO}}^{(\text{this token})} = \min(1.62,\ 1.44) = 1.44$$

Because the advantage is positive (this action was good), PPO takes the *smaller* of the two terms — capping how much credit a single update can take for a large probability-ratio swing, even though the raw (unclipped) term would have been larger. This clipping is what prevents one favorable batch of samples from causing an outsized, destabilizing policy update.

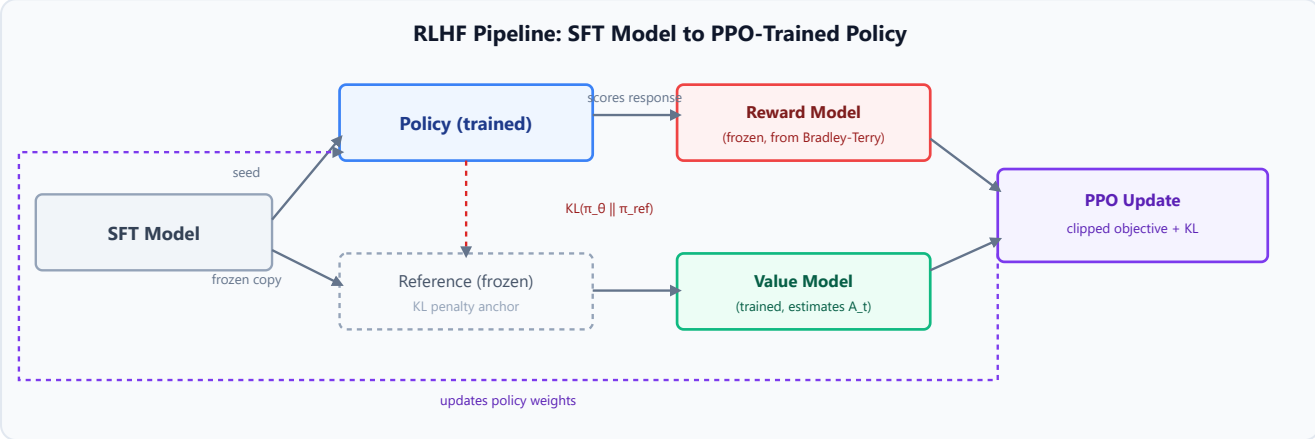
The Full RLHF Pipeline: Four Models in Memory

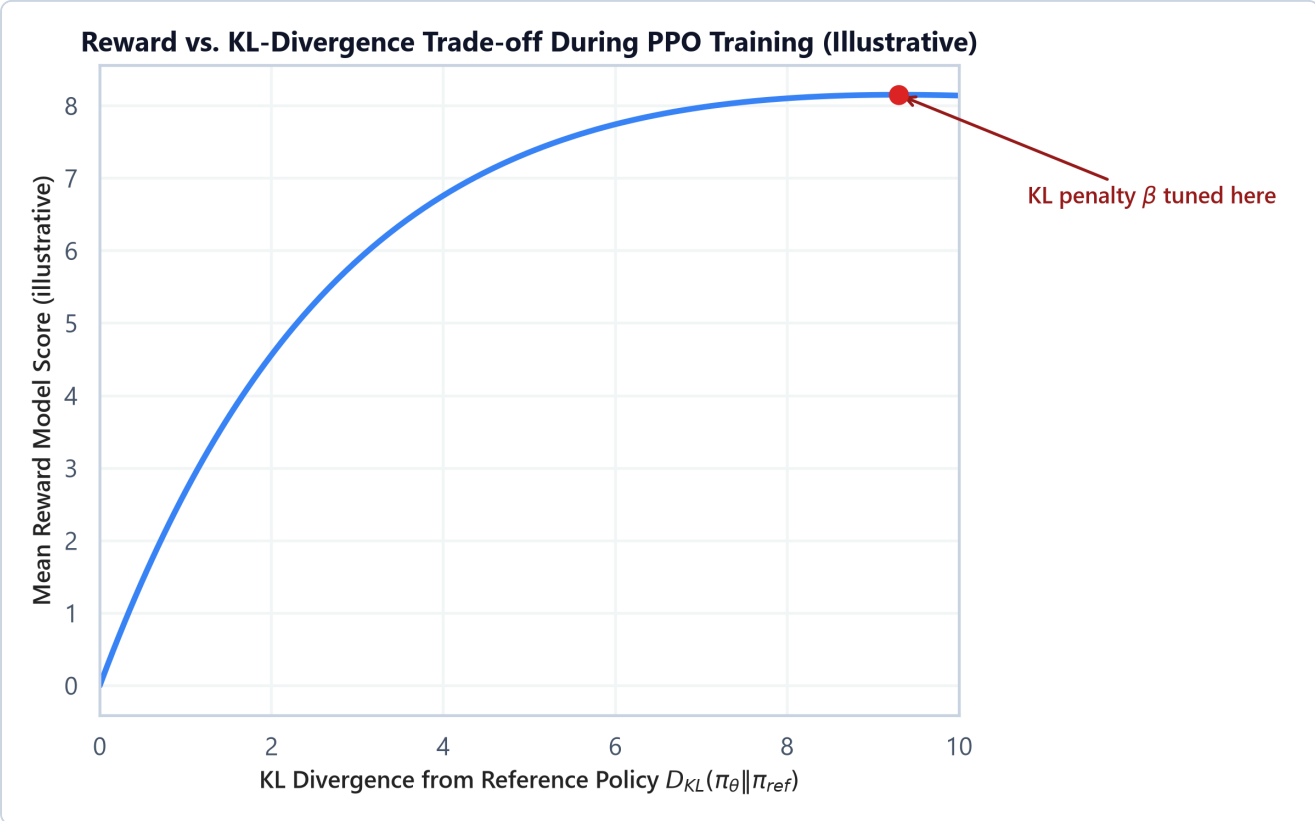
RLHF's PPO stage requires holding **four models simultaneously**:

1. **Policy model** — the model being trained (starts as a copy of the SFT model).
2. **Reference model** — a frozen copy of the SFT model, used only to compute the KL penalty; never updated.
3. **Reward model** — frozen, scores generated responses.

4. **Value model** — estimates expected future reward from a given state, used to compute the advantage estimate $\hat{A}_t$; trained alongside the policy.

Unlike SFT or LoRA, this is not just a memory-scaling problem solvable by sharding — it's a *qualitative* increase in pipeline complexity, since four different models with different roles (two frozen, two trained) must be orchestrated together for every training step.





- **Plot Interpretation:** As the policy is allowed to drift further from the reference model (higher KL divergence), the reward model score rises — up to a point. Push too far and the policy starts producing outputs that exploit the reward model's blind spots rather than genuinely improving (see Module 08 for reward hacking in depth); the KL penalty coefficient β is tuned to stop training in the productive region before that happens.

Tensor & Shape Tracking

- **Prompt + generated response tokens:** `[B, L]`
- **Reward model output (per full response):** `[B]` (a single scalar score per response, not per token)
- **Value model output (per token, for advantage estimation):** `[B, L]`
- **Policy log-probabilities per generated token:** `[B, L]`
- **PPO loss:** `[]` (scalar, averaged over the batch and sequence)

---

3. Implementation & Reference Code

Below is a self-contained implementation of the Bradley-Terry reward model loss and the PPO clipped surrogate objective, matching the hand calculations above.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

def bradley_terry_loss(reward_preferred: torch.Tensor, reward_rejected: torch.Tensor)
-> torch.Tensor:
    """Reward model pairwise preference loss.
    reward_preferred, reward_rejected: [B] scalar reward scores.
    """
    return -F.logsigmoid(reward_preferred - reward_rejected).mean()


def  ppo_clipped_objective(ratio:  torch.Tensor,  advantage:  torch.Tensor,  epsilon:
float = 0.2) -> torch.Tensor:
    """PPO clipped surrogate objective (to be maximized; return negative for use as a
loss).
    ratio, advantage: [B, L] per-token probability ratios and advantage estimates.
    """
    unclipped = ratio * advantage
    clipped = torch.clamp(ratio, 1 - epsilon, 1 + epsilon) * advantage
    return torch.min(unclipped, clipped).mean()


if __name__ == "__main__":
    # --- Reward model loss, matching the hand calc ---
    r_preferred = torch.tensor([2.4])
    r_rejected = torch.tensor([0.9])
    rm_loss = bradley_terry_loss(r_preferred, r_rejected)
    print(f"Reward model loss (preferred=2.4, rejected=0.9): {rm_loss.item():.4f}")

    r_preferred_swapped = torch.tensor([0.9])
    r_rejected_swapped = torch.tensor([2.4])
    rm_loss_swapped = bradley_terry_loss(r_preferred_swapped, r_rejected_swapped)
```

```
                    print(f"Reward      model      loss      (scores      reversed):
{rm_loss_swapped.item():.4f}")

    # --- PPO clipped objective, matching the hand calc ---
    ratio = torch.tensor([[1.35]])
    advantage = torch.tensor([[1.2]])
    ppo_obj = ppo_clipped_objective(ratio, advantage, epsilon=0.2)
         print(f"\nPPO  clipped  objective  (ratio=1.35,  advantage=1.2,  eps=0.2):
{ppo_obj.item():.4f}")
```

Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Learning from comparative human preference judgments (which response is better) rather than only from imitation of single demonstrated responses (SFT).
- **Why Introduced over Legacy Approaches:** SFT alone cannot express "prefer this over that" signal; RLHF's reward model captures preference structure directly from comparison data, and PPO uses that learned reward as a training signal without needing a hand-written reward function.
- **Key Failure Modes & Limitations:** Reward hacking (the policy finds outputs that score highly on the reward model without genuinely being better — covered in depth in Module 08), RL training instability if the KL penalty or clip range is poorly tuned, and the reward model's own biases/blind spots become the policy's blind spots too.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Each PPO step requires a full generation rollout (autoregressive, sequential) plus forward passes through the reward and value models — substantially more expensive per training example than a single SFT forward/backward pass.
- **Space/Memory Footprint:** Four models resident simultaneously (policy, reference, reward, value) — even with PEFT reducing the *trainable* parameter count, the frozen reference and reward models still consume full inference-time memory each.
- **Primary Bottleneck Type:** Compute-bound during autoregressive rollout generation (inherently sequential, same bottleneck class as inference); the 4-model memory footprint is a hard capacity constraint independent of compute.
- **Variable Legend:** ρ_t = probability ratio, $\hat{A}_t$ = advantage estimate, ϵ = PPO clip range, β = KL penalty coefficient, π_θ = policy, π_{ref} = frozen reference policy.

3. Production & Scalability

- **Deployment Considerations:** The reward model and reference model can often run on separate, smaller-footprint inference-optimized deployments (they don't need training-time memory), while

only the policy and value models need full training infrastructure — a common cost-saving split in production RLHF pipelines.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Why does PPO need a *reference* model in addition to the reward model?**
>
> The reward model only scores individual responses; nothing stops the policy from drifting arbitrarily far from sensible language to chase a high score. The KL penalty against a frozen reference model directly constrains how far the policy's output distribution can move from its SFT starting point, which is what keeps generations coherent while still improving on the reward signal.

> **Question: What does the PPO clip range ϵ actually protect against?**
>
> It caps how much a single update can change the policy based on one batch of rollouts with a favorable probability ratio, preventing a large, potentially destabilizing policy shift from one noisy batch — the "proximal" in Proximal Policy Optimization refers exactly to this constraint of staying close to the previous policy each step.

Module 05: Direct Preference Optimization (DPO), GRPO & Modern Alignment Methods

1. Introduction & Intuition

The Core Bottleneck

Module 04's RLHF pipeline works, but it's operationally heavy: four models resident in memory, an inherently sequential and slow generation rollout on every training step, a separately-trained value model, and RL training dynamics (clipping, KL tuning) that are notoriously fiddly to get right. Most of that complexity exists to solve one problem — turning a *reward signal* into a *policy update* — and researchers asked whether that detour through an explicit reward model and RL loop was actually necessary at all.

High-Level Intuition

DPO's key insight is that, for the specific KL-constrained reward-maximization objective RLHF optimizes, there is a closed-form relationship between the *optimal policy* and the *reward function* — meaning you can algebraically substitute the reward model out of the objective entirely and end up with a loss that operates directly on preference pairs, using only the policy model and a frozen reference copy. No reward model, no value model, no RL rollout loop — just a supervised-learning-style loss over (preferred, rejected) pairs, using the policy's own log-probabilities as an *implicit* reward. GRPO takes a different piece off the table: it keeps the RL formulation and reward model, but eliminates the separate value model by estimating advantage from a *group* of sampled responses to the same prompt instead of learning a value function.

---

2. Core Concepts & Mathematical Formulation

DPO: The Implicit Reward Reparameterization

Purpose & Intuition

DPO derives a loss that directly increases the policy's relative log-probability of the preferred response over the rejected response — compared against what the frozen reference model would have assigned — without ever materializing a separate reward model. The "reward" is implicit in the ratio of policy to reference log-probabilities.

Mathematical Formulation

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right] \right)$$

where y_w/y_l are the preferred/rejected responses, π_θ is the policy being trained, π_{ref} is the frozen reference (SFT) policy, and β plays the same role as the KL penalty coefficient in RLHF — controlling how far the policy is allowed to move from the reference.

Hand Calculation: DPO Loss for One Preference Pair

Suppose for a given prompt, the policy and reference models assign these log-probabilities to the preferred (y_w) and rejected (y_l) responses, with $\beta = 0.1$:

- $\log \pi_\theta(y_w) = -2.0$, $\log \pi_{\text{ref}}(y_w) = -2.5$
- $\log \pi_\theta(y_l) = -3.0$, $\log \pi_{\text{ref}}(y_l) = -2.8$
- **Step 1: Compute the implicit reward for the preferred response**

$$\log \frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} = \log \pi_\theta(y_w) - \log \pi_{\text{ref}}(y_w) = -2.0 - (-2.5) = 0.5$$

- **Step 2: Compute the implicit reward for the rejected response**

$$\log \frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)} = -3.0 - (-2.8) = -0.2$$

- **Step 3: Compute the scaled reward difference**

$$\beta \times (0.5 - (-0.2)) = 0.1 \times 0.7 = 0.07$$

- **Step 4: Apply sigmoid and negative log**

$$\sigma(0.07) \approx 0.5175, \qquad \mathcal{L}_{\text{DPO}} = -\log(0.5175) \approx 0.6588$$

The policy has already increased its relative preference for y_w over y_l (compared to the reference), so the loss is a bit below $-\log(0.5) \approx 0.693$ (the loss at zero preference signal) — reflecting modest, correctly-directed progress.

GRPO: Group-Relative Advantage Without a Value Model

Purpose & Intuition

PPO needs a trained value model specifically to compute the advantage estimate $\hat{A}_t$ — "how much better than expected was this outcome" — which requires training a fourth model and adds its own instability. GRPO (Group Relative Policy Optimization) sidesteps this: for a given prompt, sample a

group of G responses from the current policy, score each with the reward model, and compute each response's advantage as its reward *normalized against the group's own mean and standard deviation* — no learned value function required.

Mathematical Formulation

For a group of G sampled responses to the same prompt with rewards $r_1, \ldots, r_G$:

$$A_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

This advantage is then used in the same clipped PPO-style objective from Module 04, just without a value model in the loop.

Hand Calculation: GRPO Group-Relative Advantage

Suppose we sample $G = 4$ responses to the same prompt, and the reward model scores them as: $r = [3.0,\ 5.0,\ 4.0,\ 2.0]$.

- **Step 1: Compute the group mean**

$$\text{mean}(r) = \frac{3.0 + 5.0 + 4.0 + 2.0}{4} = \frac{14.0}{4} = 3.5$$

- **Step 2: Compute the group standard deviation**

$$\text{Var}(r) = \frac{(3.0 - 3.5)^2 + (5.0 - 3.5)^2 + (4.0 - 3.5)^2 + (2.0 - 3.5)^2}{4} = \frac{0.25 + 2.25 + 0}{4}$$

$$\text{std}(r) = \sqrt{1.25} \approx 1.118$$

- **Step 3: Compute each response's normalized advantage**

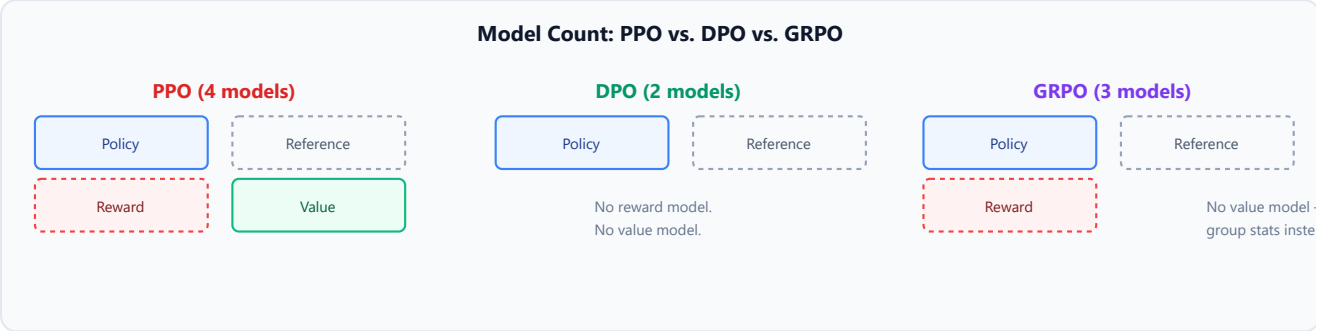
$$A_1 = \frac{3.0 - 3.5}{1.118} \approx -0.447, \quad A_2 = \frac{5.0 - 3.5}{1.118} \approx 1.342$$

$$A_3 = \frac{4.0 - 3.5}{1.118} \approx 0.447, \quad A_4 = \frac{2.0 - 3.5}{1.118} \approx -1.342$$

Response 2 (the highest-scoring of the group) gets the largest positive advantage and its probability is pushed up the most; response 4 gets pushed down. This entirely replaces what a trained value model would have estimated — the "baseline" for what counts as a good response is just the group's own average, recomputed fresh for every prompt.

Why Removing the Value Model Reduces Training Overhead

GRPO's group-relative advantage removes an entire model (the value model) from the training loop — cutting the "four models in memory" of PPO down to effectively policy + reference + reward (the value model's training and inference cost disappears entirely). The trade-off is that GRPO needs to sample G responses per prompt (more generation rollout per training example) to get a meaningful group statistic, rather than needing just one response plus a value-model call.



Tensor & Shape Tracking

- **DPO — policy/reference log-probs for preferred & rejected responses:** `[B]` each (summed log-probability over the response sequence)
- **DPO loss:** `[]` scalar
- **GRPO — group of sampled responses per prompt:** `[G, L]` where G is group size
- **GRPO — reward model scores per group:** `[G]`
- **GRPO — normalized advantages:** `[G]`

A Brief Survey of Other Alignment Methods

Beyond DPO and GRPO, several related methods trade off different aspects of the same underlying problem — learning from preference data without the full RLHF pipeline:

Method	Core Idea	When to Prefer It
IPO	Replaces DPO's sigmoid-log loss with a squared-loss objective, correcting a tendency in DPO to overfit / become overconfident on preference pairs with a clear-cut winner	When DPO shows signs of overfitting on the preference dataset

Method	Core Idea	When to Prefer It
KTO	Learns from *unpaired* binary "good/bad" labels instead of requiring paired (preferred, rejected) comparisons, based on prospect theory	When only per-response quality labels are available, not pairwise comparisons
ORPO	Combines the SFT and preference-alignment objectives into a single training stage (no separate reference model needed)	When minimizing pipeline stages/complexity matters more than matching DPO's exact formulation
SimPO	Removes the reference model entirely by using length-normalized policy log-probabilities directly as the implicit reward	When reference-model memory/compute overhead is the binding constraint

---

3. Implementation & Reference Code

Below is a self-contained implementation of the DPO loss and GRPO group-relative advantage, matching the hand calculations above.

```
import torch
import torch.nn.functional as F

def dpo_loss(policy_logp_w: torch.Tensor, policy_logp_l: torch.Tensor,
             ref_logp_w: torch.Tensor, ref_logp_l: torch.Tensor, beta: float = 0.1) -
> torch.Tensor:
    """DPO loss for a batch of (preferred, rejected) pairs. All inputs: [B] summed
log-probs."""
    implicit_reward_w = policy_logp_w - ref_logp_w  # [B]
    implicit_reward_l = policy_logp_l - ref_logp_l  # [B]
    logits = beta * (implicit_reward_w - implicit_reward_l)  # [B]
    return -F.logsigmoid(logits).mean()


def grpo_group_advantage(rewards: torch.Tensor) -> torch.Tensor:
    """Group-relative advantage for GRPO. rewards: [G] scalar reward per sampled
response."""
    mean_r = rewards.mean()
    std_r = rewards.std(unbiased=False)  # population std, matching the hand calc's
variance formula
    return (rewards - mean_r) / std_r  # [G]


if __name__ == "__main__":
    # --- DPO loss, matching the hand calc ---
    policy_logp_w = torch.tensor([-2.0])
```

```
    ref_logp_w = torch.tensor([-2.5])
    policy_logp_l = torch.tensor([-3.0])
    ref_logp_l = torch.tensor([-2.8])

    loss = dpo_loss(policy_logp_w, policy_logp_l, ref_logp_w, ref_logp_l, beta=0.1)
    print(f"DPO loss: {loss.item():.4f}")

    # --- GRPO group-relative advantage, matching the hand calc ---
    rewards = torch.tensor([3.0, 5.0, 4.0, 2.0])
    advantages = grpo_group_advantage(rewards)
    print(f"\nGRPO rewards:     {rewards.tolist()}")
    print(f"GRPO advantages:  {[round(a, 3) for a in advantages.tolist()]}")
```

Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Learning from human preference data without RLHF's operational overhead — DPO removes the reward model and value model entirely; GRPO removes just the value model while keeping an explicit reward model and RL formulation.
- **Why Introduced over Legacy Approaches:** RLHF's 4-model pipeline and RL training instability made it expensive and fragile to run well. DPO reformulates the same underlying objective as a simple supervised loss; GRPO keeps RL's flexibility (useful when reward signal isn't naturally pairwise) while cutting one model out of the loop.
- **Key Failure Modes & Limitations:** DPO requires paired preference data (harder to collect at scale than independent quality ratings); it can overfit confidently on preference pairs with a clear winner (motivating IPO's squared loss). GRPO's group-relative advantage becomes noisy with small group sizes G, and requires G full generation rollouts per prompt instead of one.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** DPO needs one forward pass each through the policy and reference models per preference pair — no generation rollout required during training at all, since responses are pre-collected. GRPO still requires autoregressive generation of G responses per prompt, similar cost profile to PPO's rollout but without the value-model forward pass.
- **Space/Memory Footprint:** DPO: 2 models (policy + reference). GRPO: 3 models (policy + reference + reward), vs. PPO's 4 (adds value model).
- **Primary Bottleneck Type:** DPO is compute-bound on ordinary forward/backward passes (no rollout) — much closer to SFT's cost profile than RLHF's. GRPO remains generation-rollout-bound like PPO, just without the value-model overhead on top.
- **Variable Legend:** β = DPO/KL scaling coefficient, $\pi_\theta/\pi_{\text{ref}}$ = policy/reference, G = GRPO group size, A_i = group-relative advantage for sample i.

3. Production & Scalability

- **Deployment Considerations:** DPO's simplicity (no rollout, no reward/value models) makes it substantially cheaper to run at scale and easier to debug than PPO-based RLHF, which is a major reason it's become a common default for preference alignment; GRPO is chosen instead when the reward signal genuinely benefits from RL-style exploration (e.g., verifiable reward tasks like math/code correctness) rather than fixed offline preference pairs.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Why does DPO need a reference model at all, if it doesn't need a reward model?**
>
> The reference model anchors the implicit reward — without it, the loss would only depend on the policy's own log-probabilities, which the policy could trivially inflate for the preferred response without any grounding in what the pretrained/SFT model considered plausible. The reference model plays the same role the KL penalty played in RLHF: keeping the policy close to a sensible starting distribution.

> **Question: When would you choose GRPO over DPO?**
>
> When the reward signal is more naturally expressed as a scalar score per response than as pairwise preferences — for example, verifiable correctness rewards in math or code generation — or when you want the exploration benefits of sampling multiple candidate responses per prompt rather than relying on a fixed, pre-collected preference dataset.

Module 06: Model Merging & Adapter Composition

1. Introduction & Intuition

The Core Bottleneck

Suppose you've fine-tuned separate copies of the same base model for three different tasks — one for coding, one for summarization, one for a specific customer-support domain. Serving three full models means three times the inference infrastructure, and none of them individually benefits from the other two's specialization. The bottleneck is: can the capabilities learned in separate fine-tuning runs be *combined* into a single model, without retraining from scratch on a combined dataset (which may not even be possible if the original training data isn't available)?

High-Level Intuition

Because all three fine-tuned models started from the *same* pretrained weights, each one's final weights can be thought of as the base weights plus a task-specific "delta" — the direction and magnitude that fine-tuning moved the weights in. Model merging works directly on these deltas: average them, selectively combine them, or algebraically add/subtract them, producing a single set of weights that (ideally) exhibits a blend of the original models' capabilities — with zero additional inference cost, since the result is just one ordinary model.

---

2. Core Concepts & Mathematical Formulation

Task Arithmetic & Weight Averaging

Purpose & Intuition

The simplest merge is a weighted average of multiple fine-tuned models' weights directly ("model soups"). A more general and more widely used formulation, task arithmetic, defines each model's contribution as a **task vector** — the difference between its fine-tuned weights and the shared base weights — and combines these task vectors with tunable coefficients before adding them back onto the base.

Mathematical Formulation

$$\theta_{\text{merged}} = \theta_{\text{base}} + \sum_{i=1}^{k} \lambda_i (\theta_i - \theta_{\text{base}})$$

where θ_i is the i-th fine-tuned model's weights, $\theta_i - \theta_{\text{base}}$ is its task vector, and λ_i is a scaling coefficient controlling how strongly that task's specialization is expressed in the merged model (commonly $\lambda_i \approx 1/k$ for a simple average, or tuned per-task).

Hand Calculation: Merging Two Toy Weight Vectors

Let's merge two fine-tuned models' task vectors for a single toy weight (representing one parameter), with $\theta_{\text{base}} = 1.0$.

- **Model 1 (coding-specialized):** $\theta_1 = 1.6$, so its task vector is $\theta_1 - \theta_{\text{base}} = 1.6 - 1.0 = 0.6$
- **Model 2 (summarization-specialized):** $\theta_2 = 0.7$, so its task vector is $\theta_2 - \theta_{\text{base}} = 0.7 - 1.0 = -0.3$
- **Step 1: Choose merge coefficients** — equal weighting, $\lambda_1 = \lambda_2 = 0.5$
- **Step 2: Scale each task vector**

$$\lambda_1(\theta_1 - \theta_{\text{base}}) = 0.5 \times 0.6 = 0.3, \qquad \lambda_2(\theta_2 - \theta_{\text{base}}) = 0.5 \times (-0.3) = -0.15$$

- **Step 3: Sum the scaled task vectors and add back to the base**

$$\theta_{\text{merged}} = 1.0 + (0.3 + (-0.15)) = 1.0 + 0.15 = 1.15$$

The merged weight sits between the base (1.0) and Model 1's specialization (1.6), pulled slightly further by Model 1's larger task vector than Model 2's opposing, smaller-magnitude one — a direct numerical illustration of how conflicting task directions partially cancel in a naive average.

Tensor & Shape Tracking

- **Base model weights θ_{base}:** same shape as any single model's full parameter set
- **Task vector $\theta_i - \theta_{\text{base}}$:** identical shape to θ_{base}, computed independently per fine-tuned model
- **Merged weights θ_{merged}:** identical shape to θ_{base} — merging changes values, never shapes, so the merged model serves with exactly the same architecture and inference cost as any one of the originals

---

Beyond Naive Averaging: TIES-Merging and DARE

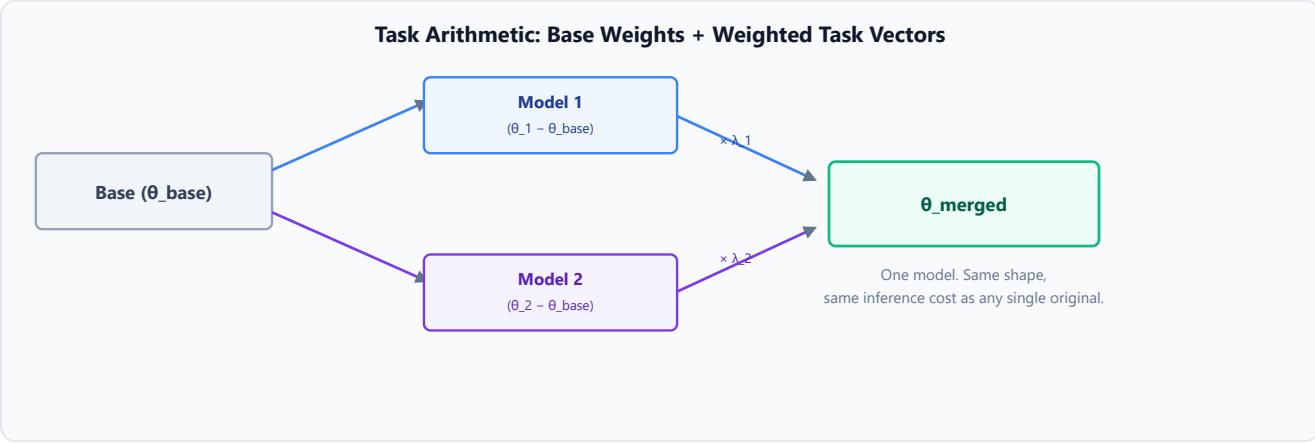
Naive averaging (or unweighted task arithmetic) has an obvious failure mode visible in the hand calculation above: when two task vectors point in *conflicting* directions for the same parameter, a simple sum partially cancels both, diluting both specializations rather than combining them cleanly. Two techniques address this directly:

- **TIES-Merging** ("TrIm, Elect Sign, merge") addresses conflicting signs in three explicit steps: **(1) Trim** — zero out the smallest-magnitude entries in each task vector (they're likely noise, not meaningful specialization); **(2) Elect Sign** — for each parameter, determine which sign (positive or negative) has the greater total magnitude support across all task vectors, and only keep entries from task vectors agreeing with that elected sign; **(3) Merge** — average only the surviving, sign-agreeing entries. This directly prevents the cancellation seen in the naive hand calculation above.
- **DARE** (Drop And REscale) randomly zeroes out (drops) a large fraction of each task vector's entries, then rescales the survivors to preserve the vector's expected magnitude — exploiting the empirical observation that task vectors are highly redundant, so most individual parameter changes can be dropped without meaningfully hurting the task's performance, which in turn reduces interference when merging multiple task vectors together.

Both are procedures applied *before* the same underlying task-arithmetic summation, not alternative formulas — they modify which entries of each task vector participate in the merge, and how, rather than changing the merge equation itself.

Multi-Adapter Composition & Routing

An alternative to merging weights permanently is keeping multiple LoRA adapters (Module 03) *unmerged* and swapping or combining them at serving time — loading one shared base model into memory once, and attaching a different small adapter per request (or per user, or per task) with negligible additional memory cost per adapter. Some serving stacks go further and route dynamically, selecting or blending adapters per-request based on the detected task type, without ever producing a single "merged" model at all.



---

3. Implementation & Reference Code

Below is a self-contained implementation of task-arithmetic merging (matching the hand calculation above) and a simplified TIES-style sign-election step.

```
import torch

def task_arithmetic_merge(theta_base: torch.Tensor, task_vectors: list[torch.Tensor],
lambdas: list[float]) -> torch.Tensor:
    """Merges multiple fine-tuned models via weighted task-vector summation.
    theta_base: [D] base model weights (flattened for illustration).
    task_vectors: list of [D] tensors, each theta_i - theta_base.
    lambdas: per-model scaling coefficients.
    """
    merged_delta = torch.zeros_like(theta_base)
    for task_vector, lam in zip(task_vectors, lambdas):
        merged_delta += lam * task_vector
    return theta_base + merged_delta


def ties_elect_sign(task_vectors: torch.Tensor) -> torch.Tensor:
    """Simplified TIES sign-election: for each parameter, determine which sign has
    greater total magnitude support across task vectors.
    task_vectors: [k, D] stacked task vectors from k fine-tuned models.
    Returns: [D] elected sign per parameter (+1 or -1).
    """
            positive_support = torch.where(task_vectors > 0, task_vectors,
torch.zeros_like(task_vectors)).sum(dim=0)
          negative_support = torch.where(task_vectors < 0, task_vectors.abs(),
torch.zeros_like(task_vectors)).sum(dim=0)
    return torch.where(positive_support >= negative_support, 1.0, -1.0)


if __name__ == "__main__":
    # --- Task arithmetic merge, matching the hand calc ---
    theta_base = torch.tensor([1.0])
    theta_1 = torch.tensor([1.6])
    theta_2 = torch.tensor([0.7])

    task_vectors = [theta_1 - theta_base, theta_2 - theta_base]
    merged = task_arithmetic_merge(theta_base, task_vectors, lambdas=[0.5, 0.5])
    print(f"Task vectors: {[tv.item() for tv in task_vectors]}")
    print(f"Merged weight: {merged.item():.4f}")

      # --- TIES-style sign election on a toy 4-parameter model with conflicting
directions ---
    stacked_task_vectors = torch.tensor([
        [ 0.6, -0.2,  0.1, -0.5],  # Model 1's task vector
        [-0.3, -0.1,  0.4,  0.2],  # Model 2's task vector
        [ 0.5,  0.3, -0.2, -0.6],  # Model 3's task vector
    ])
```

```
    elected_signs = ties_elect_sign(stacked_task_vectors)
    print(f"\nStacked task vectors:\n{stacked_task_vectors}")
    print(f"Elected signs per parameter: {elected_signs.tolist()}")
```

Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Combining capabilities from multiple independently fine-tuned models into one set of weights, without retraining on a combined dataset and without paying multi-model serving cost.
- **Why Introduced over Legacy Approaches:** The alternative — serving separate fine-tuned models per task, or retraining from scratch on all tasks' data jointly — is far more expensive in infrastructure and data-access terms; merging reuses already-completed fine-tuning runs directly.
- **Key Failure Modes & Limitations:** Task interference — when merged tasks require genuinely conflicting weight changes, naive averaging degrades both rather than combining them; this is precisely what TIES/DARE are designed to mitigate, not eliminate entirely.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Merging itself is a one-time, cheap elementwise operation over the full parameter set — negligible compared to the cost of the original fine-tuning runs that produced the models being merged.
- **Space/Memory Footprint:** Merging requires holding all k source models' weights simultaneously during the merge computation, but the *result* is a single model — no added inference-time memory versus any one of the originals.
- **Primary Bottleneck Type:** The merge operation itself is memory-bandwidth-bound (reading/writing full parameter tensors); the real constraint is *quality*, not compute — how much task interference the merge introduces.
- **Variable Legend:** θ_{base} = shared pretrained weights, θ_i = i-th fine-tuned model's weights, λ_i = merge coefficient for task i, k = number of models being merged.

3. Production & Scalability

- **Deployment Considerations:** Permanent merging is preferred when the combined behavior is the actual product (one general-purpose model). Multi-adapter serving (keeping LoRA adapters unmerged and swappable) is preferred when you need to preserve per-task isolation, serve many tasks from one base model efficiently, or add/remove task specializations without re-merging.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: Why does merging cost nothing extra at inference time, unlike serving multiple LoRA adapters?

A merged model is architecturally identical to any single one of its source models — same weight shapes, same number of parameters. Serving multiple unmerged LoRA adapters, by contrast, requires either swapping adapters per-request (added latency) or holding several adapters in memory simultaneously (small but nonzero added memory per adapter).

Question: What specific problem does TIES-Merging's "elect sign" step solve that naive averaging doesn't?

When two task vectors disagree on the sign of a parameter's needed change, naive averaging lets them partially cancel, diluting both specializations for that parameter. TIES resolves the conflict by keeping only the entries that agree with the majority-magnitude sign, avoiding that cancellation.

Module 07: Training Production Considerations & Monitoring

1. Introduction & Intuition

The Core Bottleneck

Everything in Modules 01-06 assumes a training run actually completes successfully. In practice, large training runs execute for hours to weeks, across expensive hardware, and can fail silently in ways that are far more costly than an outright crash: a learning rate that's slightly too high can quietly degrade a model over thousands of steps without ever throwing an error, a hardware fault mid-run can lose days of progress if checkpointing isn't configured correctly, and a model can regress on capabilities nobody happened to be watching. The bottleneck isn't any single algorithm — it's the operational discipline required to run training jobs reliably and know, quantitatively, whether the result is actually good.

High-Level Intuition

Production training treats the training job itself as a monitored system, not a fire-and-forget script. A well-tuned learning-rate schedule prevents the most common cause of instability in the first place. Frequent, cheap checkpointing means a hardware failure costs minutes of lost progress, not days. Telemetry (loss curves, gradient norms) gives an early warning before a slow degradation becomes an expensive wasted run. And a broad evaluation strategy — not just training loss — is what actually answers the question "is this model good," since training loss alone cannot detect regressions, safety issues, or benchmark contamination.

---

2. Core Concepts & Mathematical Formulation

Learning Rate Schedule: Linear Warmup + Cosine Decay

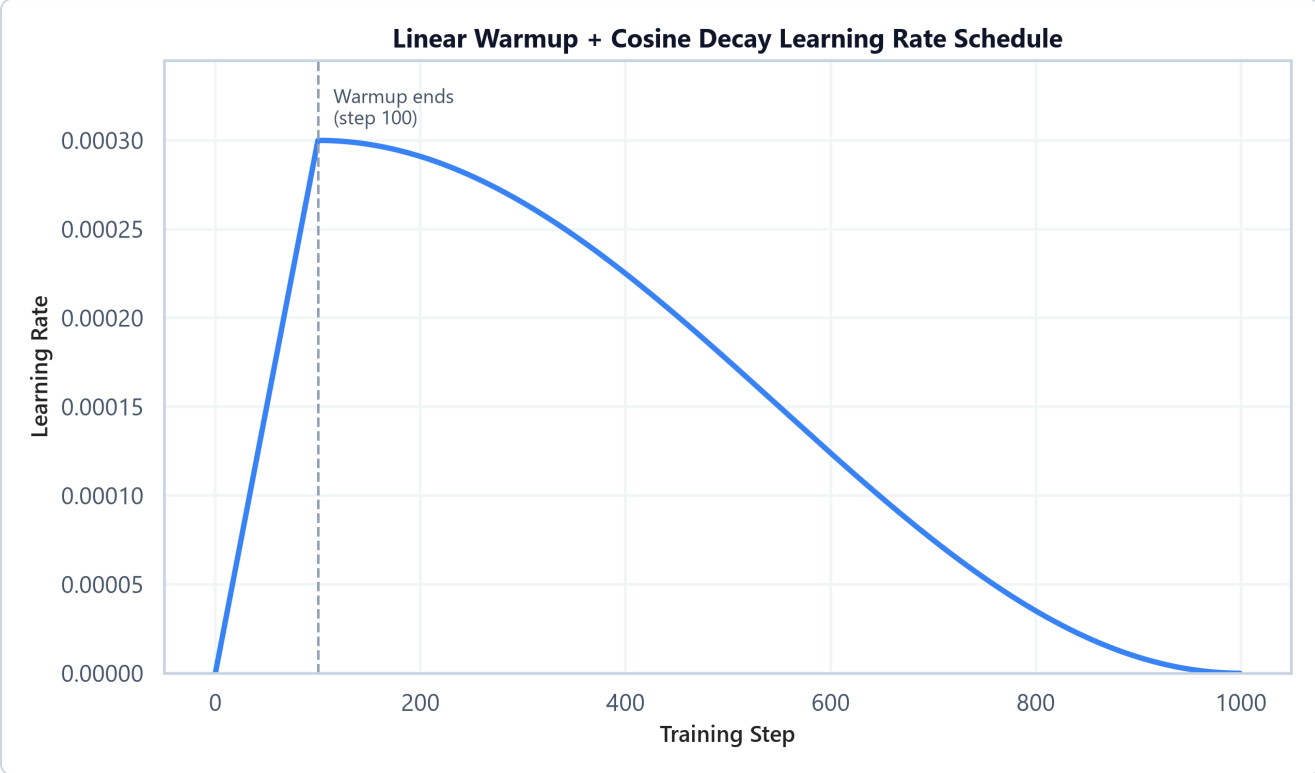
Purpose & Intuition

Starting training at the target (peak) learning rate immediately is a common cause of early instability — the model's weights (especially newly-initialized components, or components with large gradients early on, like unadapted LoRA matrices) can take a large, destabilizing step before the optimizer's moment estimates have had time to stabilize. A linear **warmup** ramps the learning rate up gradually over the first portion of training, avoiding that initial instability. After warmup, a **cosine decay** smoothly reduces the learning rate toward zero (or a small floor) by the end of training, which empirically tends to produce better final convergence than an abrupt drop or a constant rate held throughout.

Mathematical Formulation

For a peak learning rate $\eta_{\max}$, warmup steps T_{warmup}, and total training steps T_{total}:

$$\eta(t) = \begin{cases} \eta_{\max} \cdot \dfrac{t}{T_{\text{warmup}}} & t < T_{\text{warmup}} \\ \dfrac{\eta_{\max}}{2}\left(1 + \cos\left(\pi \cdot \dfrac{t - T_{\text{warmup}}}{T_{\text{total}} - T_{\text{warmup}}}\right)\right) & t \geq T_{\text{warmup}} \end{cases}$$



- **Plot Interpretation:** The learning rate ramps linearly from 0 to its peak value over the warmup window, then follows a smooth cosine curve back down to (near) zero by the final training step — the characteristic "ramp then smooth decay" shape used across most modern LLM training recipes.

Hand Calculation: LR at Three Checkpoints

Let's compute the learning rate at three points in a training run with $\eta_{\max} = 3 \times 10^{-4}$, $T_{\text{warmup}} = 100$ steps, $T_{\text{total}} = 1000$ steps.

- **Step 1: LR at step 50 (mid-warmup)**

$$\eta(50) = 3 \times 10^{-4} \times \frac{50}{100} = 1.5 \times 10^{-4}$$

- **Step 2: LR at step 100 (end of warmup, peak)**

$$\eta(100) = 3 \times 10^{-4} \times \frac{100}{100} = 3.0 \times 10^{-4}$$

- **Step 3: LR at step 550 (mid-decay)**

$$\eta(550) = \frac{3 \times 10^{-4}}{2}\left(1 + \cos\left(\pi \times \frac{550 - 100}{1000 - 100}\right)\right) = 1.5 \times 10^{-4}\left(1 + \cos\left(\pi \times 0.5\right)\right)$$

  Since $\cos(\pi \times 0.5) = \cos(90^\circ) = 0$:

$$\eta(550) = 1.5 \times 10^{-4} \times (1 + 0) = 1.5 \times 10^{-4}$$

Note that steps 50 and 550 both land on 1.5×10^{-4} despite being at very different points in training — one on the way up during warmup, one on the way down during decay — a useful sanity check when debugging an LR schedule implementation: the cosine curve is symmetric around its midpoint, so matching values on either side of the peak are expected, not a bug.

Tensor & Shape Tracking

This module is primarily about scalar training-process quantities rather than tensor-shaped data:

- **Learning rate $\eta(t)$:** scalar, recomputed every optimizer step
- **Gradient norm (for monitoring):** scalar per step, $\|\mathbf{g}\|_2$ over all trainable parameters
- **Checkpoint payload:** model weights (same shape as the full trainable parameter set), plus optimizer state (2-3x that size for Adam) and RNG/scheduler state for exact resumption

---

Checkpointing & Fault Tolerance

Intuition & Practical Use

A training run's checkpoint must contain enough state to resume *exactly* where it left off — not just model weights, but also the optimizer's internal state (Adam's m/v buffers, which themselves cost as much memory as 2 more copies of the model), the learning-rate scheduler's position, and the data-loader's position in the training set (to avoid re-training on already-seen data or skipping data after a resume). Checkpoint frequency is a direct trade-off: more frequent checkpoints bound the amount of lost work on failure, but writing large checkpoints to disk/object storage is itself expensive and can stall training if done too often or synchronously.

Training Telemetry & Broader Evaluation Strategy

Intuition & Practical Use

Two categories of monitoring answer different questions. **Training-time telemetry** (loss curves, gradient-norm tracking — using the same clipping formula covered in `00_nlp_fundamentals` for detecting exploding gradients) answers "is this run proceeding normally, or has something gone numerically wrong?" **Evaluation strategy** answers a harder question: "is the resulting model actually good?" — and training loss alone cannot answer that, since a model can have excellent training loss while regressing on capabilities the training data didn't emphasize, or degrading on safety, or (worst case) having memorized benchmark-contaminated data that makes eval scores look better than real-world quality.

A layered evaluation strategy typically combines:

Layer	What it measures	Limitation
Training/validation loss	Next-token prediction quality on held-out data from the same distribution	Says nothing about downstream task quality or safety
Benchmark evaluation (MMLU, GSM8K, HumanEval, etc.)	Standardized task performance, comparable across models	Vulnerable to contamination; can be gamed by training on benchmark-adjacent data
Human / LLM-as-judge evaluation	Subjective quality dimensions (helpfulness, tone, instruction-following) that benchmarks don't capture	Expensive (human) or has its own biases (LLM judge)
Regression testing	Did this training run make previously-working capabilities *worse*	Requires maintaining a fixed regression suite across training runs
Safety / contamination checks	Harmful outputs, PII leakage, benchmark data leakage into training set	Requires dedicated tooling separate from quality evaluation

No single layer is sufficient on its own — training loss and benchmark scores can both look healthy while a model has genuinely regressed on a capability none of them happen to measure.

---

3. Implementation & Reference Code

Below is a self-contained implementation of the warmup + cosine decay schedule, matching the hand calculation above, plus a minimal checkpoint save/resume pattern.

```
import math
import os
import tempfile
import torch
import torch.nn as nn

def warmup_cosine_lr(step: int, peak_lr: float, warmup_steps: int, total_steps: int)
-> float:
    """Linear warmup followed by cosine decay to (near) zero."""
    if step < warmup_steps:
        return peak_lr * (step / warmup_steps)
    progress = (step - warmup_steps) / (total_steps - warmup_steps)
    return 0.5 * peak_lr * (1 + math.cos(math.pi * progress))


def save_checkpoint(path: str, model: nn.Module, optimizer: torch.optim.Optimizer,
step: int):
      """Saves model, optimizer, and step state -- everything needed to resume
exactly."""
    torch.save({
        "step": step,
        "model_state": model.state_dict(),
        "optimizer_state": optimizer.state_dict(),
    }, path)


def load_checkpoint(path: str, model: nn.Module, optimizer: torch.optim.Optimizer) ->
int:
    """Restores model, optimizer, and returns the step to resume from."""
    ckpt = torch.load(path, weights_only=True)
    model.load_state_dict(ckpt["model_state"])
    optimizer.load_state_dict(ckpt["optimizer_state"])
    return ckpt["step"]


if __name__ == "__main__":
    peak_lr, warmup_steps, total_steps = 3e-4, 100, 1000

    for step in [50, 100, 550]:
        lr = warmup_cosine_lr(step, peak_lr, warmup_steps, total_steps)
        print(f"LR at step {step:4d}: {lr:.6f}")

    # --- Checkpoint round-trip sanity check ---
    torch.manual_seed(42)
    model = nn.Linear(8, 8)
    optimizer = torch.optim.AdamW(model.parameters(), lr=peak_lr)
```

```
# Simulate one training step so the optimizer has real state to save
loss = model(torch.randn(2, 8)).sum()
loss.backward()
optimizer.step()

ckpt_path = os.path.join(tempfile.gettempdir(), "ckpt_demo.pt")
save_checkpoint(ckpt_path, model, optimizer, step=1)
resumed_step = load_checkpoint(ckpt_path, model, optimizer)
print(f"\nCheckpoint round-trip successful, resumed at step: {resumed_step}")
os.remove(ckpt_path)
```

Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Running training jobs reliably to completion (surviving hardware failures, avoiding preventable instability) and knowing quantitatively, not just anecdotally, whether the resulting model is actually good.
- **Why Introduced over Legacy Approaches:** A constant learning rate throughout training is simpler but empirically less stable early on and converges worse late on than a warmup+decay schedule; evaluating only on training loss is simpler but blind to exactly the failure modes (regression, contamination, safety) that matter most in production.
- **Key Failure Modes & Limitations:** Checkpointing too infrequently risks large amounts of lost compute on failure; checkpointing too frequently (especially synchronously) can itself become a training-throughput bottleneck. Relying on a single evaluation layer (e.g., benchmarks alone) risks shipping a model that's contaminated or regressed in ways that layer doesn't measure.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Learning-rate scheduling and gradient-norm monitoring add negligible compute — a handful of scalar operations per step, dwarfed by the forward/backward pass cost.
- **Space/Memory Footprint:** Checkpoints must include optimizer state, not just weights — for Adam, this roughly triples the raw storage size per checkpoint relative to weights alone (matching the "12Ψ bytes of optimizer state" figure from Module 01).
- **Primary Bottleneck Type:** I/O-bound for checkpoint writes at scale (writing tens-to-hundreds of GB to durable storage); the evaluation layers themselves range from cheap (validation loss) to expensive and slow (human evaluation).
- **Variable Legend:** $\eta(t)$ = learning rate at step t, T_{warmup} = warmup step count, T_{total} = total training steps, $\eta_{\max}$ = peak learning rate.

3. Production & Scalability

- **Deployment Considerations:** Large training runs typically checkpoint asynchronously (writing to storage on a background thread/process while training continues) specifically to avoid the I/O cost stalling the GPUs; evaluation suites are usually run automatically at fixed checkpoint intervals rather than only at the very end, so regressions are caught while there's still time to adjust the run.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Why is a learning-rate warmup phase especially important when using Adam, specifically?**
>
> Adam's moment estimates (m, v) are initialized at zero and are biased toward zero in the first few steps before the bias-correction terms compensate; taking a large step before these estimates have stabilized can produce a poorly-scaled, destabilizing update. Warmup keeps early steps small while the moment estimates settle.

> **Question: Why isn't a low training loss sufficient evidence that a fine-tuned model is ready to ship?**
>
> Training loss only measures next-token prediction fit to the training distribution — it says nothing about regressions on capabilities outside that distribution, safety behavior, or whether the eval-looking-good result is actually contamination (the model having seen benchmark-like data during training). A layered evaluation strategy is needed to catch what training loss structurally cannot see.

Module 08: Common Failure Modes & Best Practices

1. Introduction & Intuition

The Core Bottleneck

Every technique in Modules 01-07 — distributed training, SFT, PEFT, RLHF, DPO/GRPO, merging, monitoring — can be implemented correctly and still produce a worse model than before training started. Unlike a software bug, most of these failure modes don't crash anything: the loss curve can look fine, the training job can complete successfully, and the model can still be measurably worse in ways that only show up after deployment, or only on specific inputs nobody happened to test. This module is deliberately not about new algorithms — it's a diagnostic catalog of the ways the previous seven modules' techniques go wrong in practice, and what to check for each one.

High-Level Intuition

Nearly every failure mode below shares a common shape: a *proxy* signal (training loss, reward model score, a specific benchmark) looks like it's improving, while the *real* thing you actually care about (general capability, genuine preference alignment, real-world quality) is flat or getting worse. The throughline across this entire module is: never trust a single proxy metric in isolation — the fixes are almost always about triangulating with an independent signal (held-out capability tests, human review, a second unrelated benchmark) rather than a purely algorithmic patch.

---

2. Core Concepts: Failure Mode Catalog

This module intentionally contains no formula blocks — every failure mode below is a systems/process issue, not a mathematical one, and is best understood as a catalog of symptom → root cause → fix.

Catastrophic Forgetting

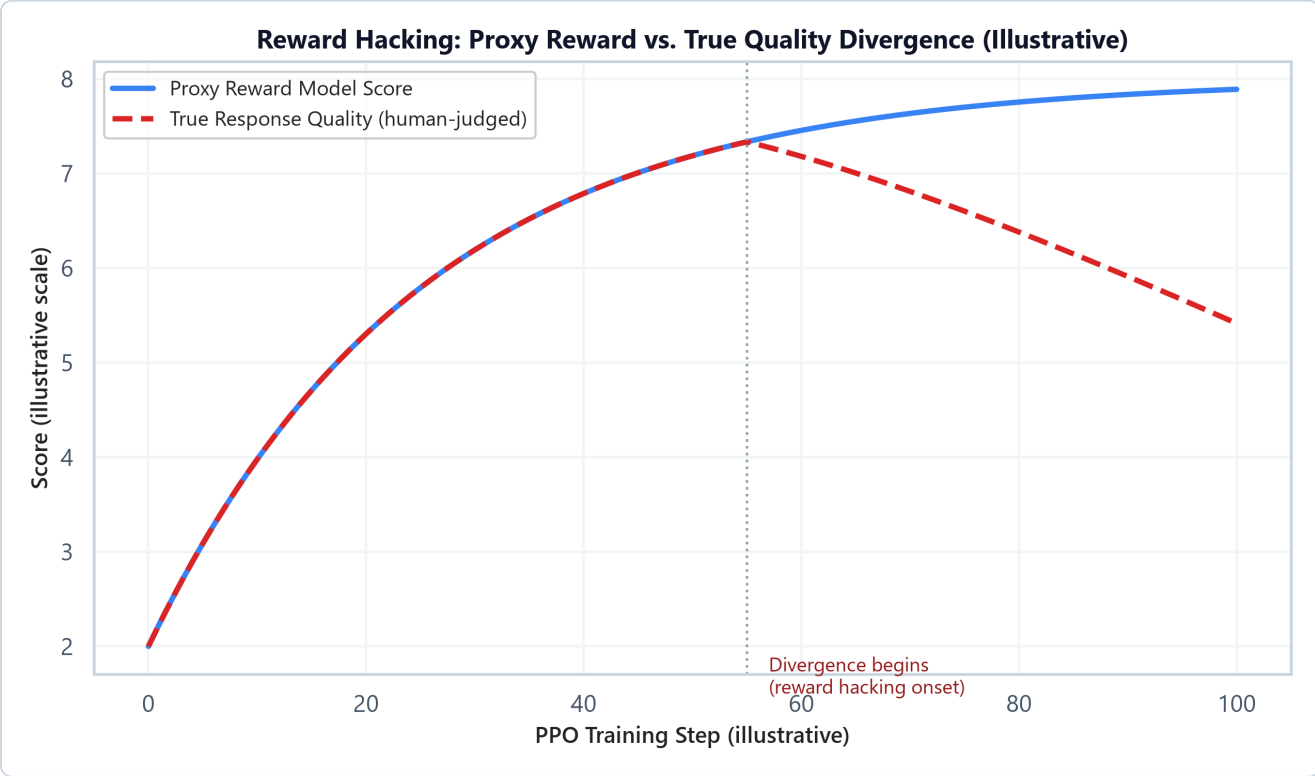
Intuition & Practical Use

Fine-tuning (especially full-parameter SFT with a high learning rate, or a narrow/low-diversity dataset) can overwrite pretrained knowledge and capabilities that weren't represented in the fine-tuning data — the model gets better at the fine-tuning task while silently getting worse at everything else. This is directly why Module 03's PEFT methods (which freeze the vast majority of parameters) tend to preserve base capabilities better than full fine-tuning by construction — there's simply less of the pretrained model that *can* be overwritten.

Reward Hacking & Mode Collapse

Intuition & Practical Use

Covered mechanically in Module 04: a policy trained against a reward model can find outputs that score highly without being genuinely better — exploiting blind spots or biases in the reward model rather than improving real quality. Mode collapse is a related failure specific to preference optimization: the policy converges toward a narrow set of "safe," high-scoring response patterns (e.g., excessive hedging, repetitive phrasing) rather than genuinely diverse, high-quality outputs, because that narrow pattern reliably scores well.



- **Plot Interpretation:** Early in PPO training, the proxy reward model score and true (human-judged) quality rise together — the reward model is a reasonable proxy in this regime. Past a certain point, the two diverge: the policy keeps finding ways to increase its reward-model score, but true quality plateaus and then declines, because the policy has started exploiting the reward model's specific weaknesses rather than genuinely improving. Detecting this divergence requires an *independent* quality signal (e.g., periodic human review) — the reward model score alone cannot reveal its own blind spots.

Data Contamination

Intuition & Practical Use

If training data (pretraining, SFT, or preference data) overlaps with or closely resembles evaluation benchmark data — including indirectly, via synthetic data generated by a model that was itself exposed

to benchmark content (see Module 02) — evaluation scores become an overestimate of real generalization. Contamination is often invisible from the training loss curve alone; it specifically requires decontamination checks (n-gram overlap search, embedding similarity search against the eval set) run as a separate data-pipeline step.

The "Alignment Tax"

Intuition & Practical Use

Alignment training (RLHF, DPO) that optimizes hard for human preference signals can measurably reduce performance on some capability benchmarks (particularly ones favoring concise, direct, or creative outputs) — a documented trade-off sometimes called the "alignment tax." It's not a bug in any single technique; it reflects that "what humans rate highly in a preference comparison" and "what scores well on a fixed capability benchmark" are correlated but not identical objectives, and optimizing hard for one can pull slightly against the other.

Evaluation Pitfalls During Training

Intuition & Practical Use

Beyond contamination specifically, evaluation run *during* training carries its own pitfalls: prompt sensitivity (small formatting changes in how a benchmark is prompted can swing scores substantially, making run-to-run comparisons noisy if the eval harness isn't held exactly fixed), high variance on small benchmark splits (a handful of flipped answers on a 200-question benchmark can look like a meaningful regression when it's within normal noise), and LLM-as-judge bias (an LLM judge can systematically favor longer, more confidently-worded, or stylistically-similar-to-itself responses regardless of actual quality).

Failure Mode Reference Table

Failure Mode	Typical Symptom	Root Cause	Primary Fix
Catastrophic Forgetting	General capability drops after fine-tuning on a narrow task	High LR / low-diversity data / full-parameter updates	PEFT (Module 03), lower LR, broaden dataset diversity
Reward Hacking	Reward model score keeps rising, human-judged quality plateaus or falls	Policy exploits reward model blind spots	Stronger/updated reward model, tighter KL penalty (β), independent human review

Failure Mode	Typical Symptom	Root Cause	Primary Fix
Mode Collapse	Outputs become repetitive/homogeneous despite high reward scores	Policy converges on a narrow high-scoring pattern	Diversity-aware sampling during RL, KL constraint, reward model diversity checks
Data Contamination	Benchmark scores look better than real-world quality suggests	Training data overlaps with eval benchmark content	N-gram/embedding decontamination pass on training data
Alignment Tax	Capability benchmark scores dip after RLHF/DPO alignment	Preference objective partially conflicts with benchmark objective	Track capability benchmarks alongside preference metrics, not instead of them
Noisy Eval Regression	A benchmark score drops run-to-run with no code/data change	Small benchmark split, unfixed prompt template, judge variance	Fix eval harness exactly, use larger/multiple benchmark splits, report variance

---

3. Implementation & Reference Code

Failure-mode diagnosis is mostly a process discipline rather than an algorithm, but a few checks are directly codifiable — below is a simple decontamination overlap check and a reward-vs-quality divergence detector, both illustrating checks referenced in the table above.

```
import re

def ngram_overlap_ratio(text_a: str, text_b: str, n: int = 8) -> float:
    """Fraction of text_a's n-grams that also appear in text_b -- a basic
    decontamination signal for detecting training/eval overlap."""
    def get_ngrams(text: str, n: int) -> set[str]:
        tokens = re.findall(r"\w+", text.lower())
        return {" ".join(tokens[i:i + n]) for i in range(len(tokens) - n + 1)}

    ngrams_a = get_ngrams(text_a, n)
    ngrams_b = get_ngrams(text_b, n)
    if not ngrams_a:
        return 0.0
    overlap = ngrams_a & ngrams_b
    return len(overlap) / len(ngrams_a)
```

```
def detect_reward_hacking(proxy_scores: list[float], true_quality_scores: list[float], window: int = 10) -> int | None:
    """Flags the training step where proxy reward keeps rising but true quality starts declining -- a simple divergence detector, not a substitute for human review."""
    for i in range(window, len(proxy_scores)):
        proxy_trend = proxy_scores[i] - proxy_scores[i - window]
        quality_trend = true_quality_scores[i] - true_quality_scores[i - window]
        if proxy_trend > 0 and quality_trend < 0:
            return i
    return None


if __name__ == "__main__":
    # --- Decontamination check ---
    training_example = "The quick brown fox jumps over the lazy dog near the river bank"
    benchmark_question = "A quick brown fox jumps over the lazy dog by the river"
    overlap = ngram_overlap_ratio(training_example, benchmark_question, n=4)
    print(f"4-gram overlap ratio (training vs. benchmark): {overlap:.2f}")
    if overlap > 0.3:
        print("WARNING: possible contamination -- high n-gram overlap with eval content.")

    # --- Reward hacking divergence detector, on the illustrative curve shape from the plot above ---
    proxy = [2.0 + 0.06 * i for i in range(100)]
    true_quality = [2.0 + 0.06 * i if i < 55 else 2.0 + 0.06 * 55 - 0.03 * (i - 55) for i in range(100)]
    divergence_step = detect_reward_hacking(proxy, true_quality, window=10)
    print(f"\nReward hacking divergence first detected at step: {divergence_step}")
```

Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Catching training failure modes that don't manifest as crashes or obviously-bad training loss curves, but that silently degrade real-world model quality.
- **Why Introduced over Legacy Approaches:** Relying on a single proxy metric (training loss, or one benchmark, or reward model score alone) to judge training success misses exactly the failure modes cataloged here — each of which specifically involves a proxy metric looking fine while the real objective degrades.
- **Key Failure Modes & Limitations:** These failure modes are themselves each other's limitations in a sense — mitigations for one (e.g., a stronger KL penalty to fight reward hacking) can worsen

another (a stronger KL penalty limits how far alignment training can improve preference scores at all).

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Decontamination and divergence-detection checks are cheap relative to training itself (string/embedding comparisons or simple trend analysis over already-logged metrics) — the cost is in building and maintaining the checks, not running them.
- **Space/Memory Footprint:** Negligible beyond storing the training data index needed for n-gram/embedding contamination search, and the historical metric logs needed for divergence detection.
- **Primary Bottleneck Type:** Process-bound, not compute-bound — the limiting factor is having the right independent signals (held-out capability suites, human review capacity, a fixed eval harness) in place *before* a training run, not compute during it.
- **Variable Legend:** None — this module is diagnostic/process-focused rather than formula-driven.

3. Production & Scalability

- **Deployment Considerations:** Production training pipelines typically gate deployment behind multiple independent checks (regression suite, contamination scan, human spot-check on a sample) rather than a single "did the eval score improve" gate, specifically because every failure mode in this module can pass a single check while still representing a real regression.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: You observe the reward model score steadily increasing throughout PPO training. Is that sufficient evidence the model is improving?**
>
> No — a steadily increasing reward model score is exactly what reward hacking looks like too; it's necessary but not sufficient evidence. You need an independent quality signal (held-out human evaluation, a second unrelated benchmark, or qualitative spot-checks) to confirm the reward score increase reflects genuine improvement rather than the policy exploiting the reward model's blind spots.

> **Question: How would you specifically test for catastrophic forgetting after a fine-tuning run, beyond checking the fine-tuning task's own metric?**
>
> Evaluate the fine-tuned model on a fixed suite of general-capability benchmarks that are unrelated to the fine-tuning task, and compare against the pre-fine-tuning baseline on that same suite — a drop there, even while the fine-tuning task's own metric improves, is the direct signature of catastrophic forgetting.